

**UNIVERSIDAD MAYOR DE SAN ANDRÉS
CARRERA DE INFORMÁTICA
FACULTAD DE CIENCIAS PURAS Y NATURALES**



**INF-263
DESARROLLO DE APLICACIONES MULTIMEDIA**

**“PROYECTO GRUPAL: Digitalización de Trámites Universitarios y
Desarrollo de Aplicaciones Multimedia Interactivas”**

ESTUDIANTES:

ESCOBAR ANGEL DANIEL
ESQUIVEL CASTILLO VIVIAN ROXANA

DOCENTE:

Ph.D. MOISES MARTIN SILVA CHOQUE

LA PAZ -BOLIVIA
2026

1. INTRODUCCIÓN

La transformación digital ha generado nuevas oportunidades para optimizar procesos administrativos y mejorar la interacción entre los usuarios y los sistemas de información. En el ámbito universitario, la incorporación de tecnologías web y multimedia permite modernizar procedimientos tradicionales, reducir tiempos de atención y ofrecer servicios más accesibles para la comunidad estudiantil.

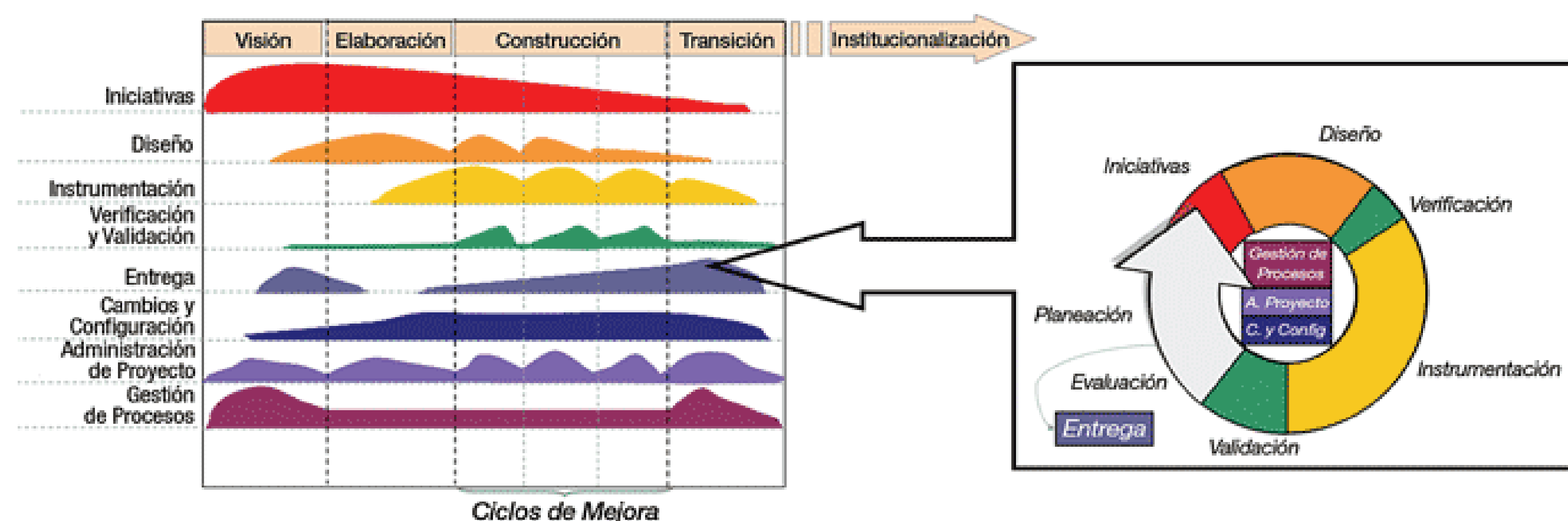
En el presente proyecto se desarrollaron diversas aplicaciones orientadas a integrar conceptos de gestión de procesos, desarrollo web, modelado tridimensional y realidad virtual. La primera parte consistió en la digitalización de trámites universitarios mediante la implementación de flujos de trabajo para la inscripción de materias y la solicitud de certificados académicos, utilizando PHP y archivos JSON como mecanismos de gestión y almacenamiento de información.

Asimismo, se desarrolló una experiencia multimedia basada en Unity, donde múltiples personajes ejecutan una coreografía sincronizada con música dentro de un entorno tridimensional exportado para su visualización mediante tecnología WebGL. Complementariamente, se aplicaron técnicas de fotogrametría para construir un avatar digital a partir de fotografías reales de un integrante del grupo, permitiendo su visualización interactiva a través de una plataforma web.

El proyecto integra diferentes herramientas y tecnologías modernas con el propósito de demostrar la aplicación práctica de conceptos relacionados con la gestión de procesos, programación web, gráficos computacionales y producción multimedia interactiva.

2. METODOLOGÍA UTILIZADA

El desarrollo del proyecto se realizó mediante una metodología incremental basada en la división de actividades en módulos independientes. Inicialmente se efectuó el análisis de los requerimientos establecidos para cada una de las actividades propuestas, identificando los objetivos, funcionalidades y herramientas necesarias para su implementación.



En la etapa de diseño se elaboraron diagramas de flujo para representar los procesos de inscripción de materias y solicitud de certificados, definiendo los roles participantes, las validaciones requeridas y las transiciones entre actividades. Posteriormente, dichos procesos fueron implementados mediante una arquitectura basada en PHP y archivos JSON para la gestión dinámica de los flujos de trabajo.

Para el desarrollo multimedia se empleó Unity como plataforma principal de creación de entornos tridimensionales. Los personajes y animaciones fueron obtenidos mediante Mixamo, adaptados al sistema Humanoid de Unity y organizados mediante controladores de animación para lograr la sincronización de las coreografías.

En la actividad de fotogrametría se utilizó Meshroom para procesar fotografías capturadas de un integrante del grupo, generando un modelo tridimensional texturizado que posteriormente fue integrado en una plataforma web interactiva para su visualización.

Finalmente, se realizaron pruebas funcionales de cada módulo con el propósito de verificar el correcto funcionamiento de los formularios, flujos de trabajo, animaciones y visualizadores tridimensionales antes de su publicación en la plataforma web final.

3. ACTIVIDAD GRUPAL A: DIGITALIZACIÓN DE TRÁMITES UNIVERSITARIOS (UMSA)

3.1 DESCRIPCIÓN GENERAL

Como parte del proyecto se desarrolló una propuesta de digitalización de trámites universitarios de la Universidad Mayor de San Andrés (UMSA), con el objetivo de **optimizar procesos administrativos mediante formularios web y gestión digital de información**. Los trámites seleccionados fueron la **inscripción de materias** y la **solicitud de certificados académicos**.

La solución fue implementada utilizando PHP para la lógica de procesamiento y archivos JSON para el almacenamiento de información, evitando el uso de sistemas gestores de bases de datos tradicionales. Esta arquitectura permitió registrar, consultar y actualizar datos de manera dinámica, manteniendo una estructura ligera y fácilmente portable.

Asimismo, se diseñaron diagramas de flujo para representar cada proceso, identificando actores involucrados, validaciones, decisiones y resultados finales.

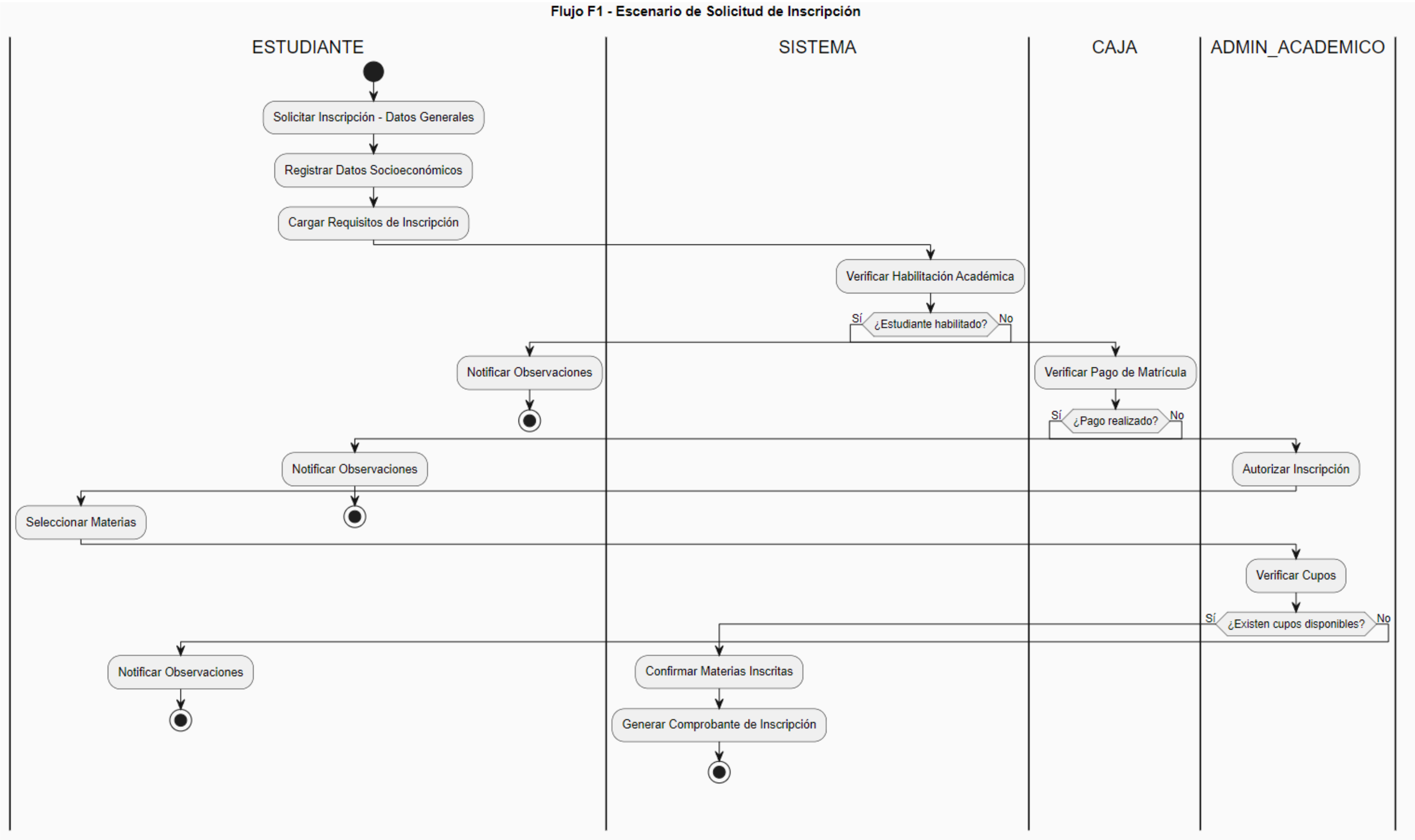
3.2 TRÁMITE 1: INSCRIPCIÓN DE MATERIAS

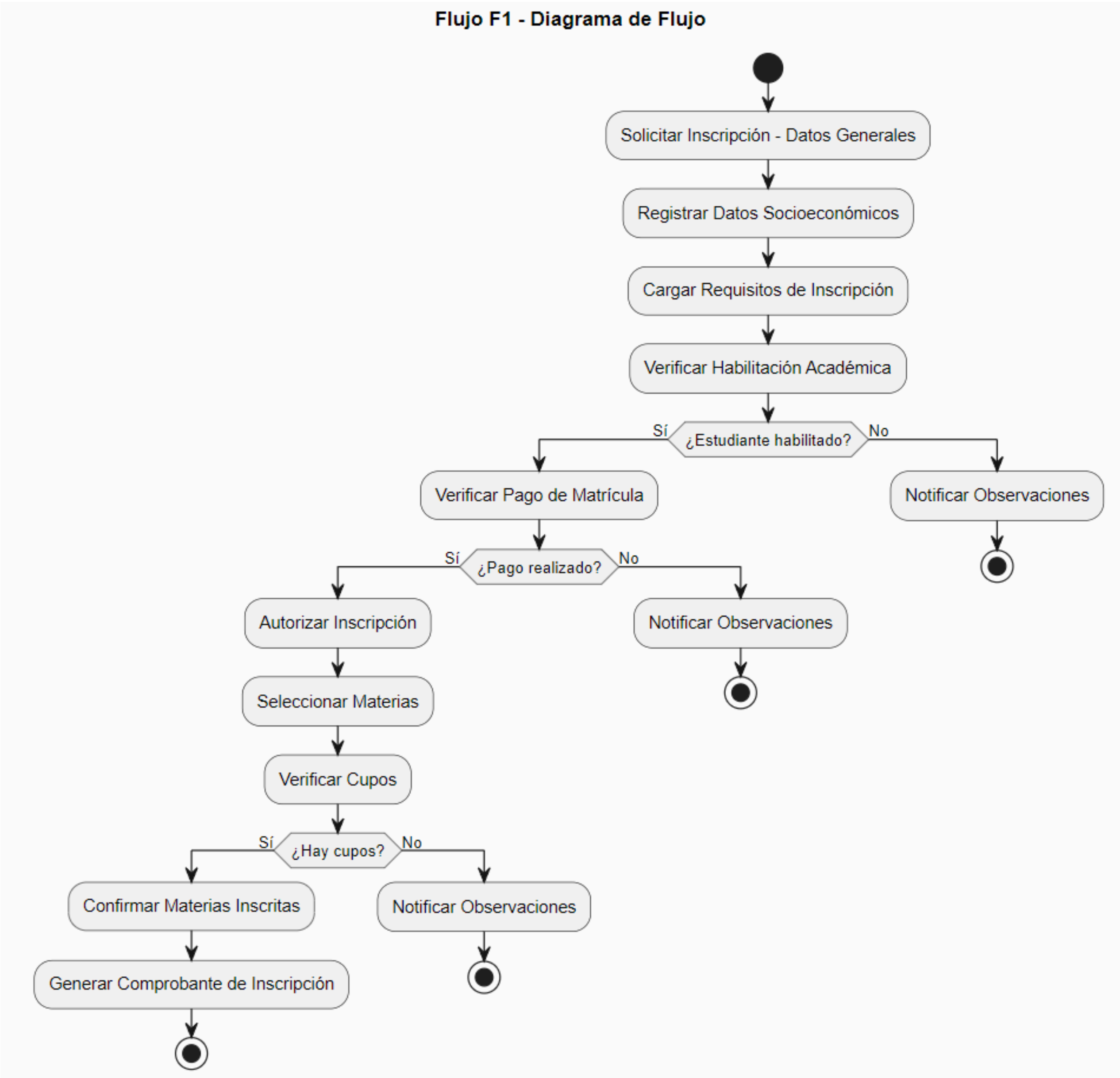
El proceso de inscripción de materias fue modelado mediante un flujo de trabajo compuesto por distintas etapas que simulan el procedimiento académico real. Inicialmente, el estudiante registra sus datos generales, posteriormente completa información socioeconómica y carga los requisitos necesarios para la inscripción.

Una vez recibida la información, el sistema realiza una verificación de habilitación académica para determinar si el estudiante cumple con las condiciones necesarias para continuar el proceso. En caso de existir observaciones, el estudiante recibe una notificación correspondiente. Si la habilitación es aprobada, el trámite continúa con la verificación del pago de matrícula por parte del área de caja.

Posteriormente, un administrador académico autoriza la inscripción y el estudiante puede seleccionar las materias deseadas. Luego se verifica la disponibilidad de cupos en las asignaturas seleccionadas. Cuando todas las validaciones son satisfactorias, el sistema confirma las materias inscritas y genera automáticamente un comprobante digital de inscripción.

Este flujo permitió representar de manera estructurada las interacciones entre estudiante, sistema, personal de caja y administración académica.





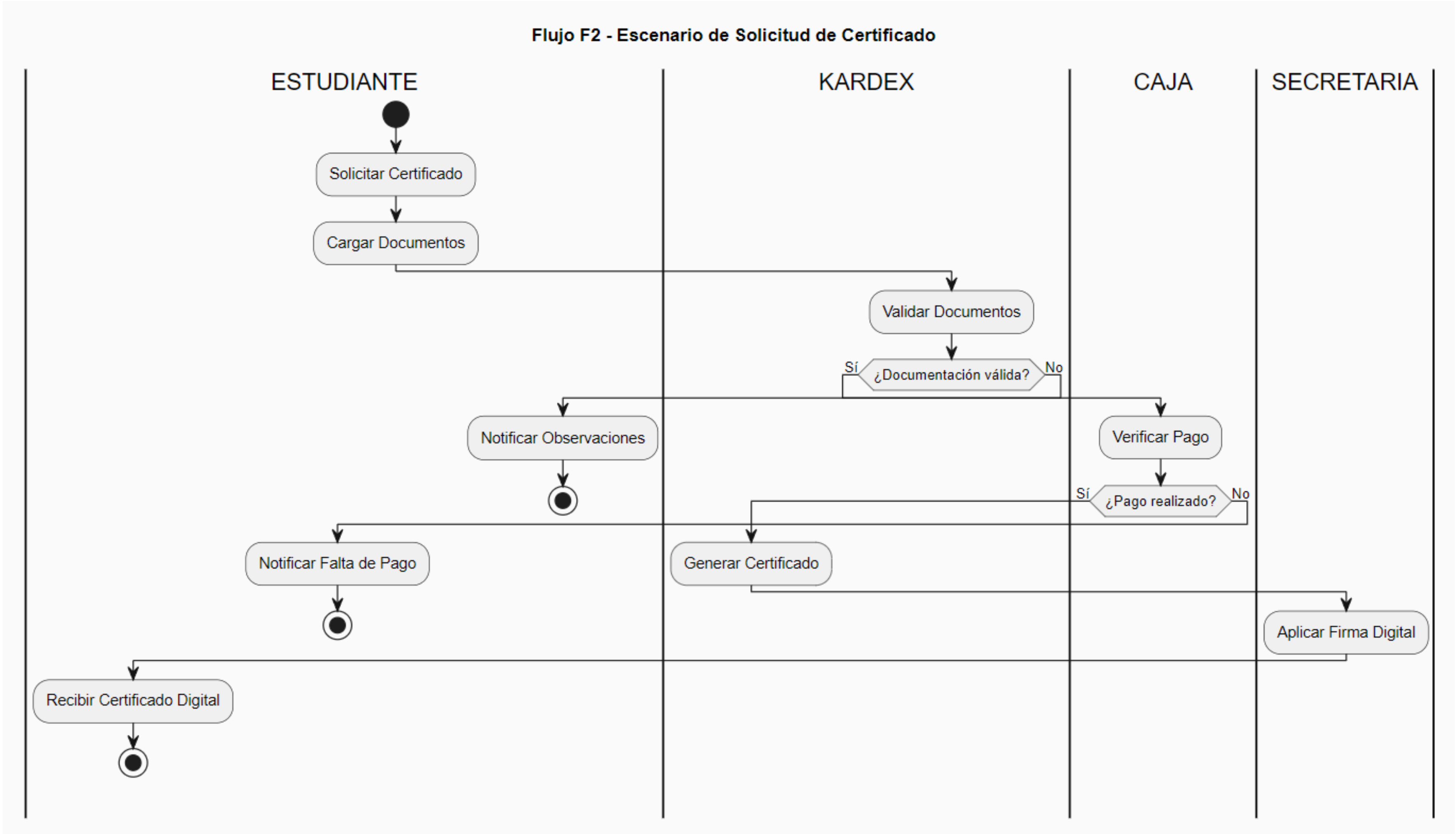
3.3 TRÁMITE 2: SOLICITUD DE CERTIFICADOS

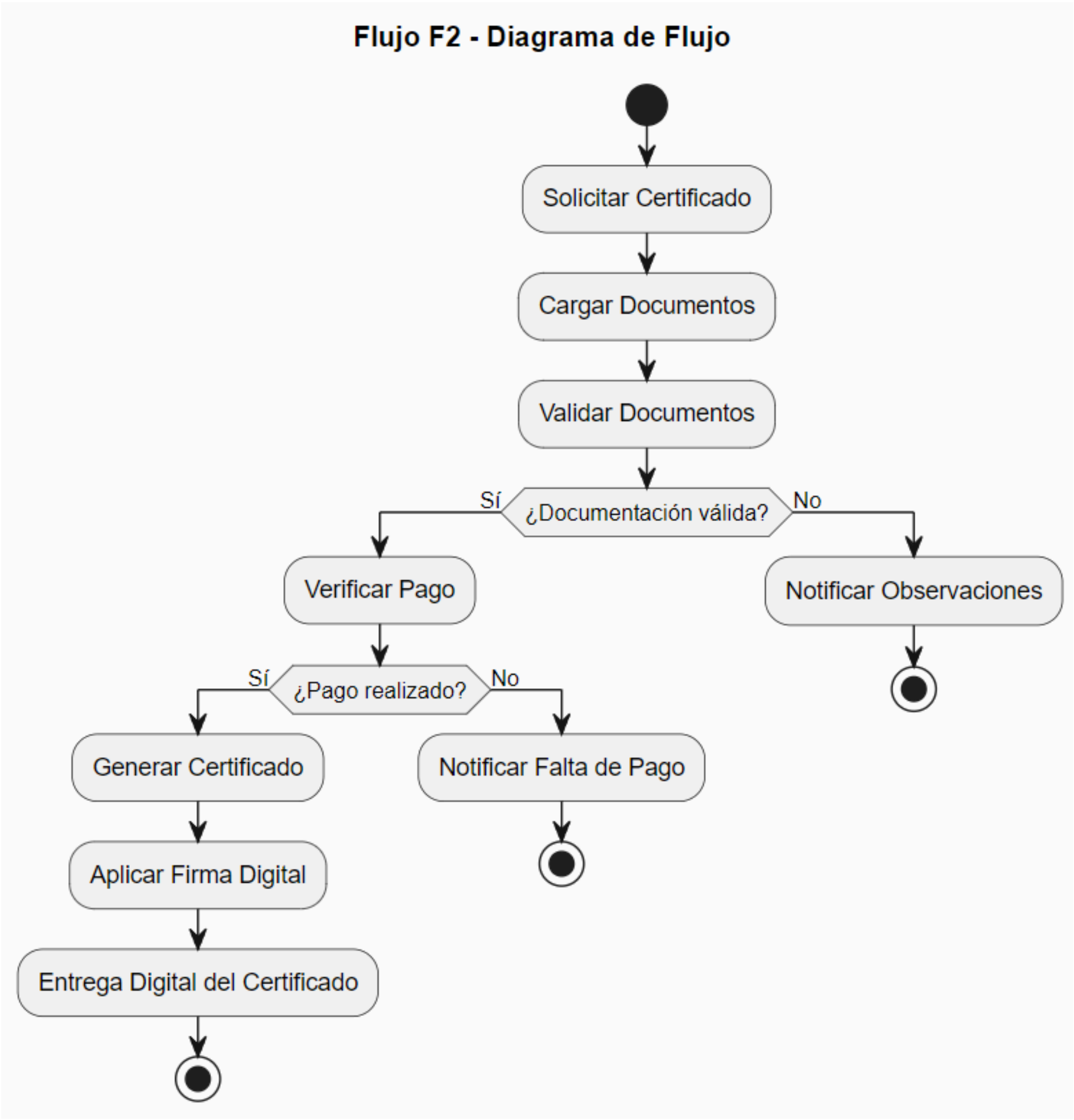
El segundo trámite implementado corresponde a la solicitud digital de certificados académicos. El proceso inicia cuando el estudiante registra la solicitud y adjunta la documentación requerida.

Posteriormente, el área de Kardex realiza la validación de los documentos presentados. Si se detectan inconsistencias o documentación incompleta, el sistema genera observaciones para que el estudiante pueda corregirlas. En caso de aprobación, se procede a verificar el pago correspondiente mediante el área de caja.

Una vez confirmado el pago, Kardex genera el certificado solicitado y lo remite a secretaría para la aplicación de la firma digital. Finalmente, el documento es entregado al estudiante mediante medios digitales, completando el trámite sin necesidad de procesos presenciales.

La implementación de este flujo permitió automatizar el seguimiento del trámite y reducir tiempos de atención mediante validaciones digitales.





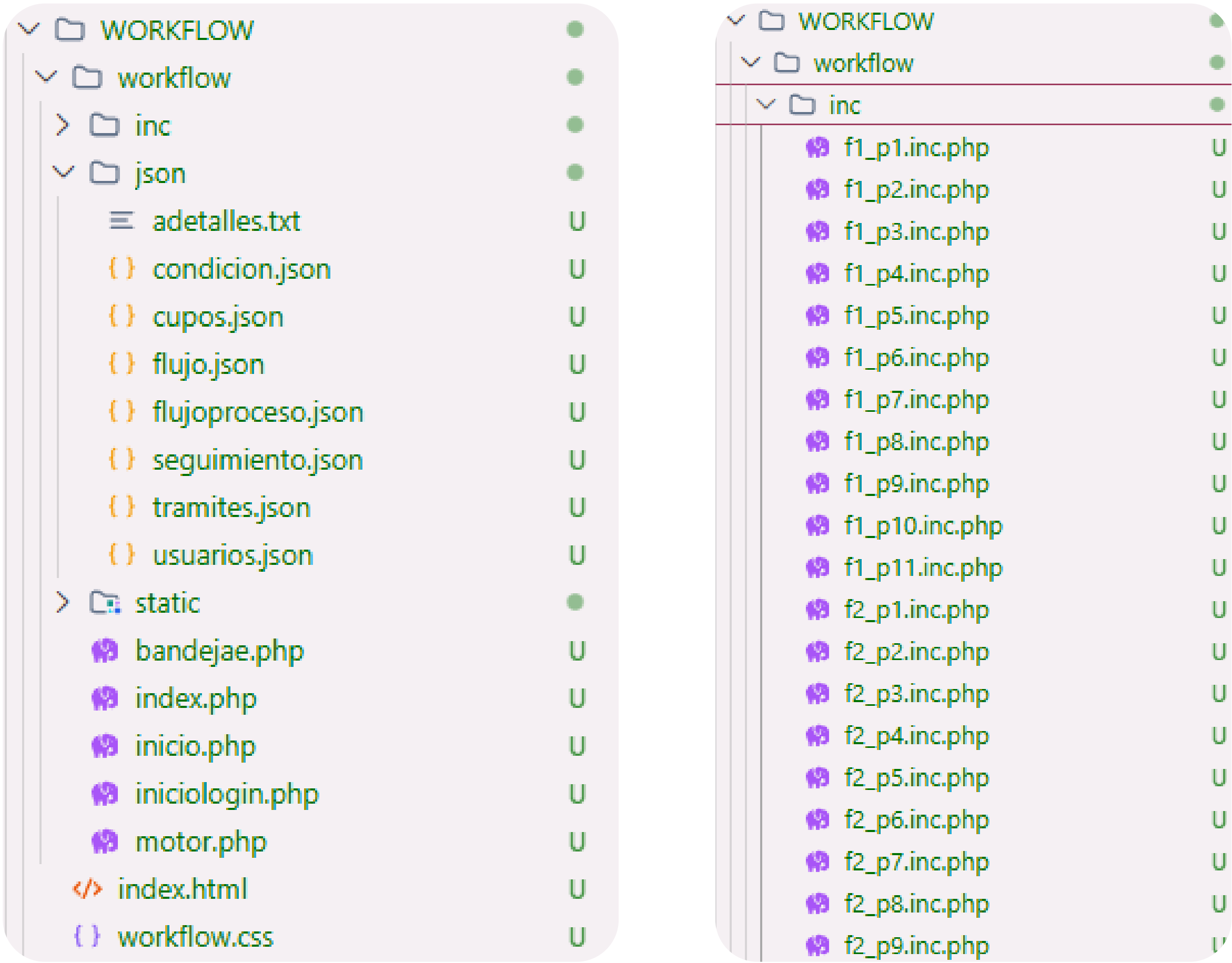
3.4 IMPLEMENTACIÓN BPM MEDIANTE ARCHIVOS JSON

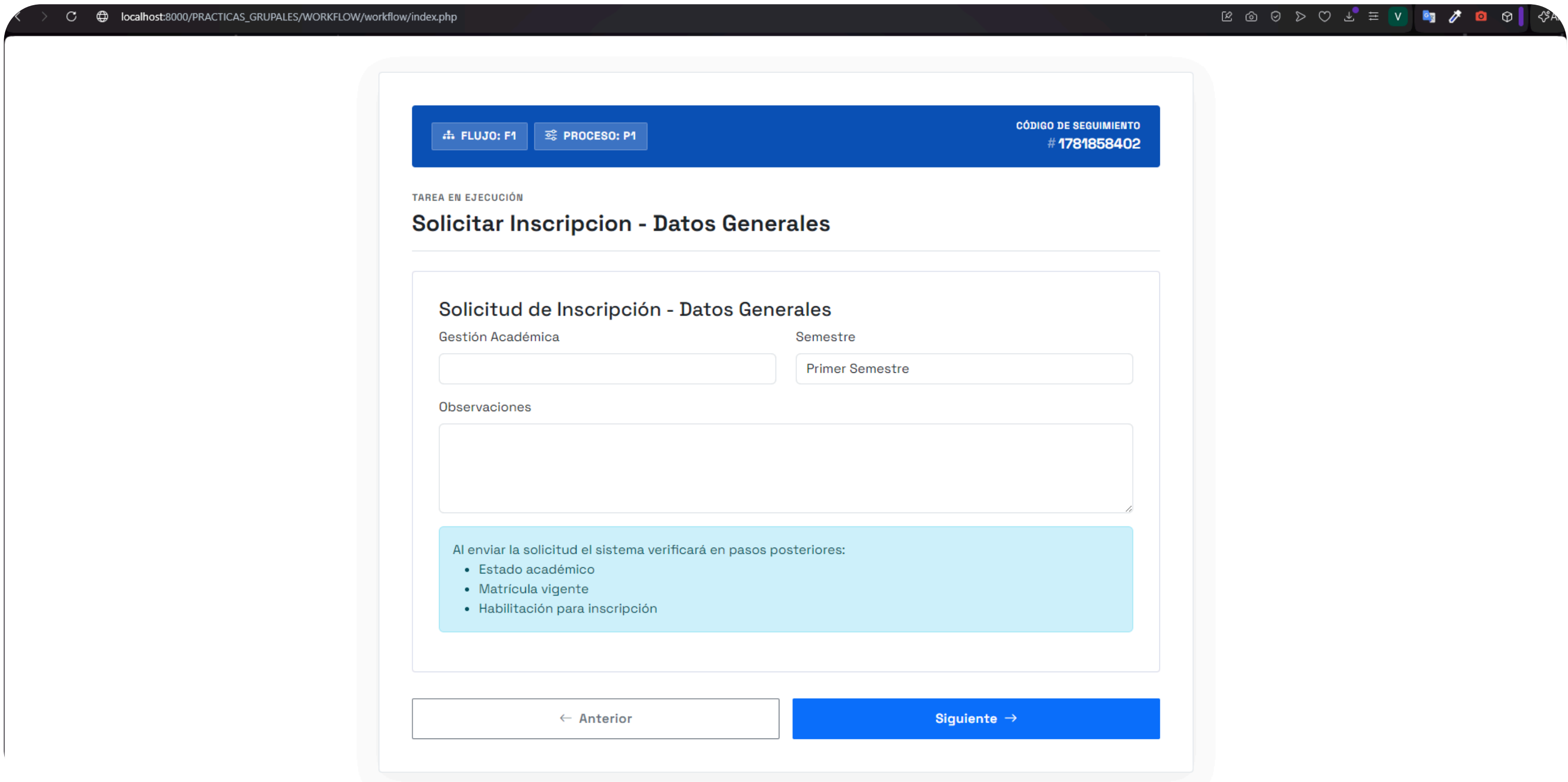
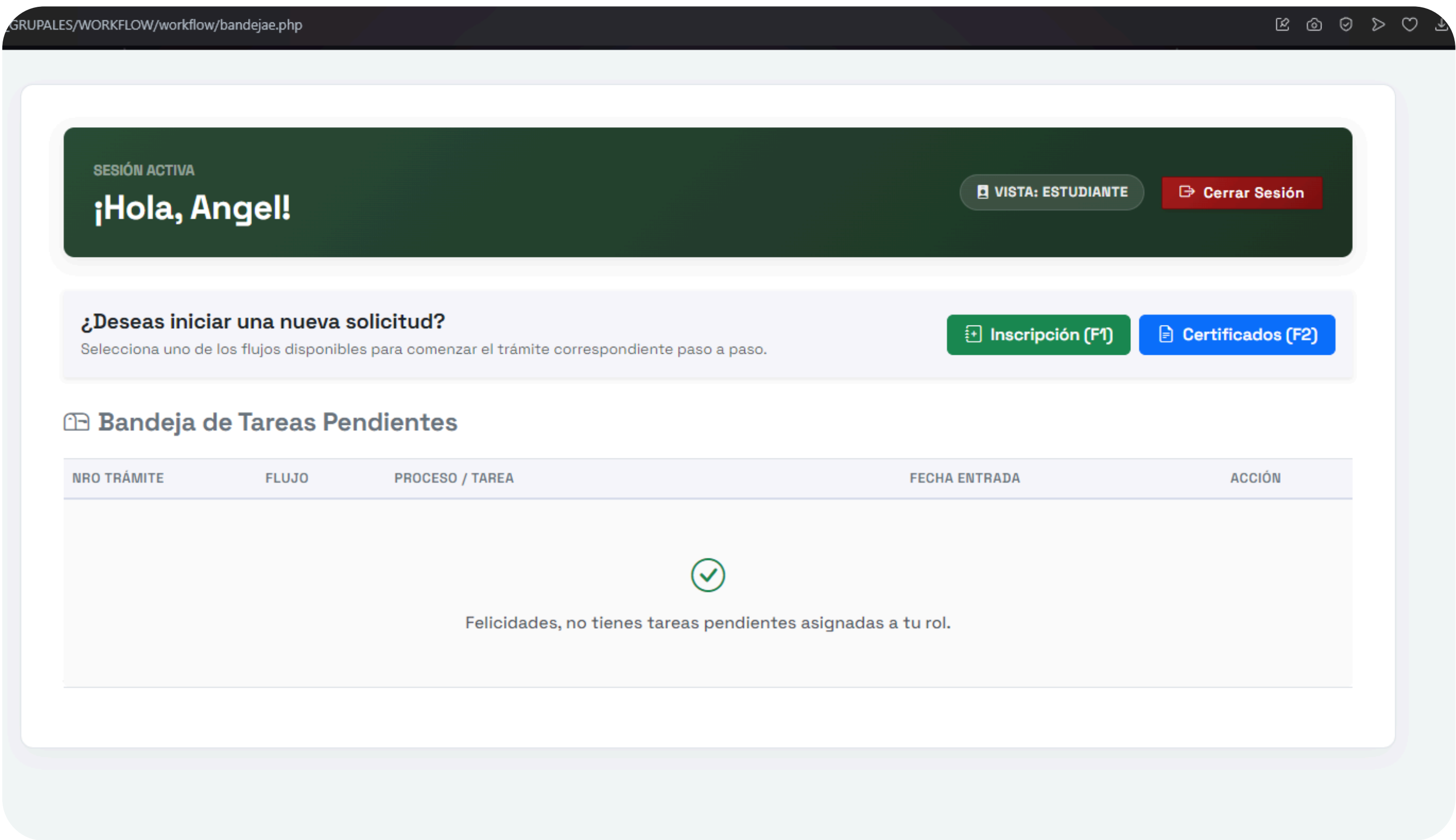
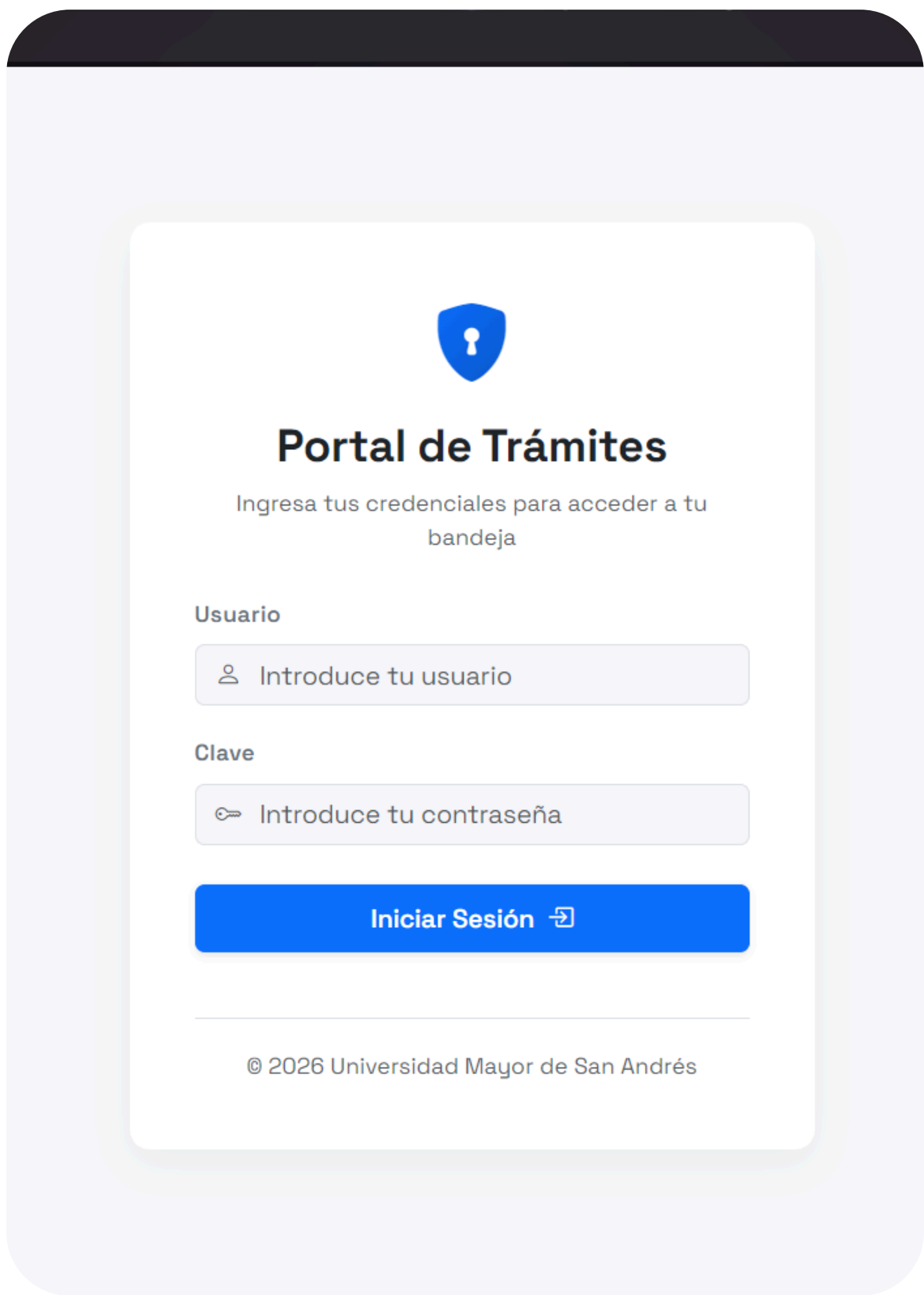
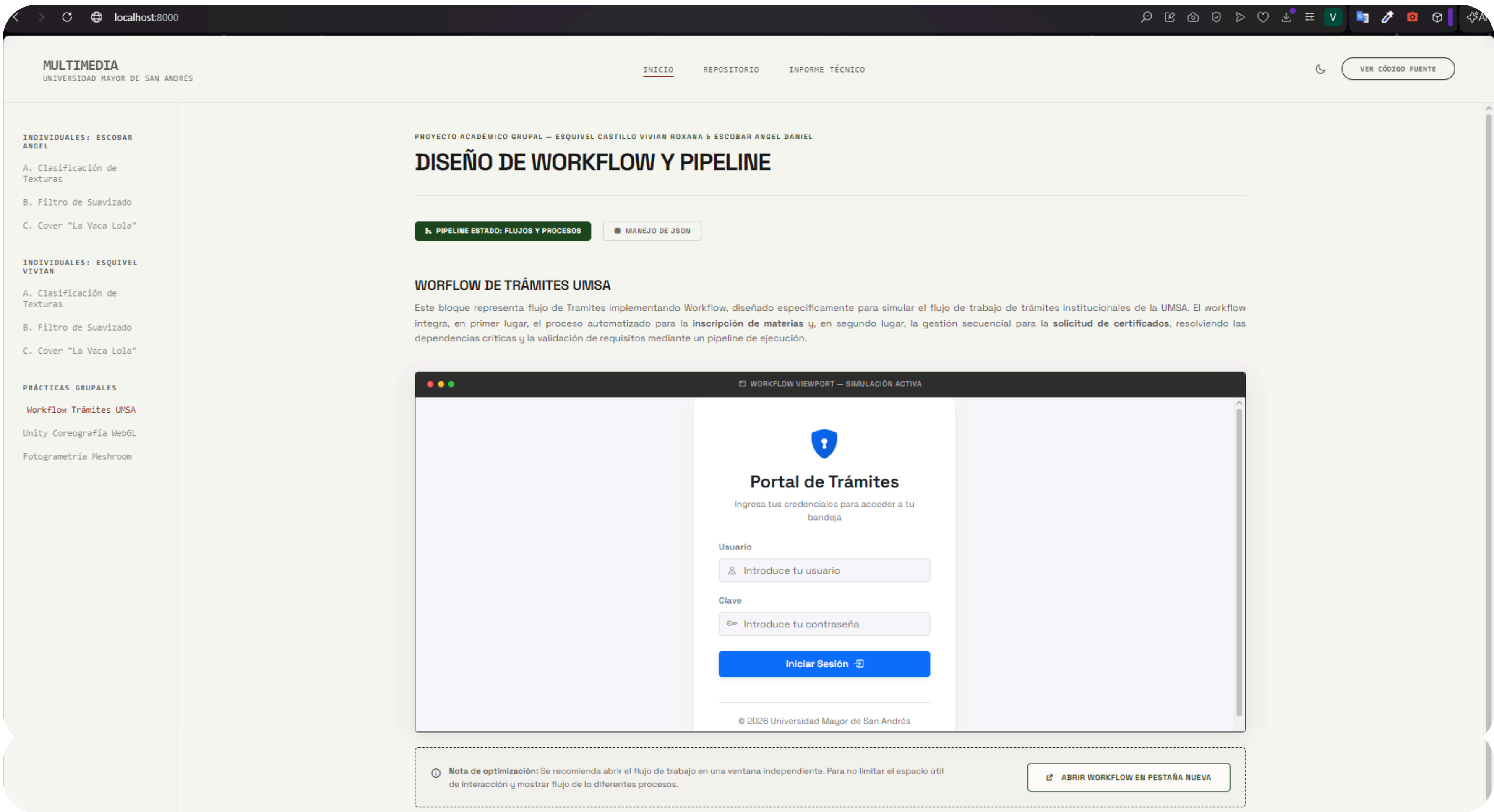
La implementación del sistema se basó en conceptos de **Business Process Management (BPM)**, utilizando archivos JSON como mecanismo principal para definir y controlar los flujos de trabajo. En lugar de emplear una base de datos tradicional, toda la información relacionada con los procesos fue almacenada y gestionada mediante estructuras JSON, permitiendo una administración flexible y dinámica de los trámites. La arquitectura del proyecto fue organizada en diferentes módulos. **La carpeta "json" almacena la definición de los flujos, procesos, roles y transiciones entre actividades.** Cada proceso contiene información referente al flujo al que pertenece, nombre de la actividad, pantalla asociada, rol responsable de ejecutarla y el siguiente paso dentro del procedimiento.

La carpeta "inc" contiene los formularios y componentes de interfaz correspondientes a cada etapa del trámite. Estos archivos son cargados dinámicamente por el motor principal del sistema, permitiendo mostrar al usuario la actividad que corresponde según el estado actual del flujo.

Por otra parte, archivos como **"motor.php"** se encargan de **interpretar la definición almacenada en los JSON y controlar la navegación entre procesos**, mientras que las páginas principales gestionan el acceso de usuarios y la interacción con los formularios del sistema.

Este enfoque permitió separar la lógica del negocio de la presentación visual, facilitando la modificación de los procesos sin necesidad de realizar cambios significativos en el código fuente. Además, la utilización de archivos JSON proporcionó una solución ligera, portable y adecuada para representar flujos de trabajo académicos de manera estructurada.

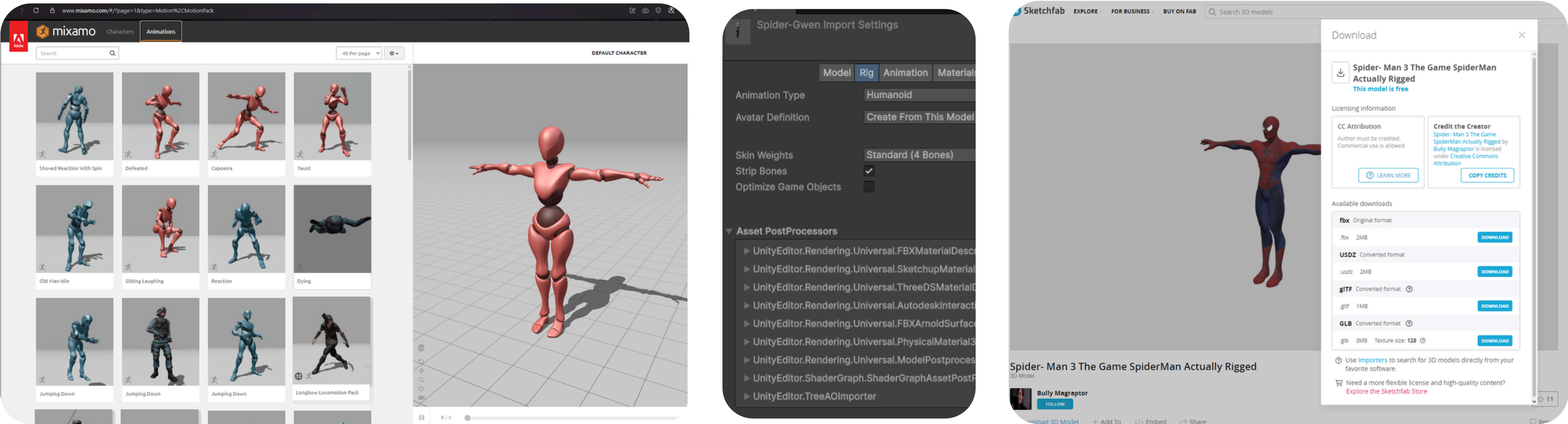




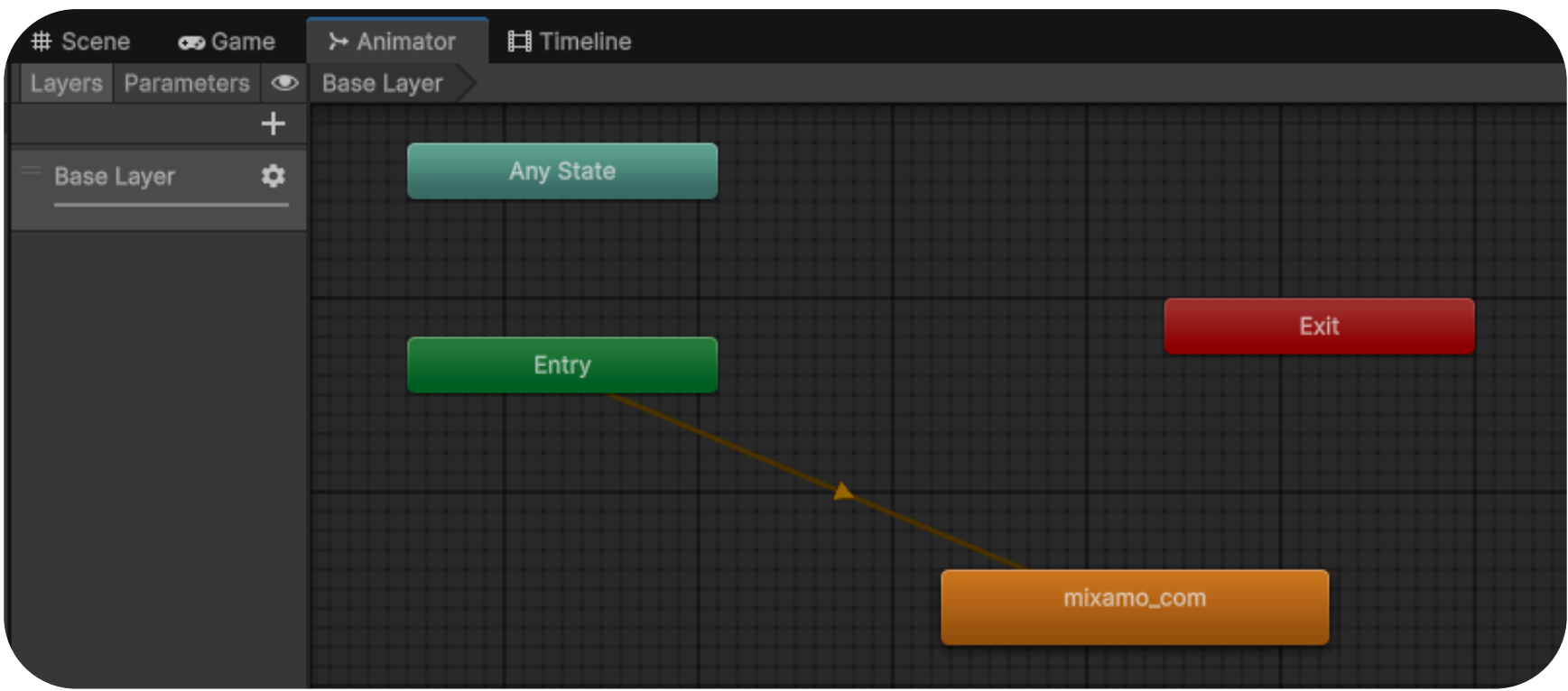
4. ACTIVIDAD GRUPAL B: DESARROLLO 3D Y FOTOGAMETRIA

4.1 ANIMACIÓN SINCRONIZADA EN UNITY

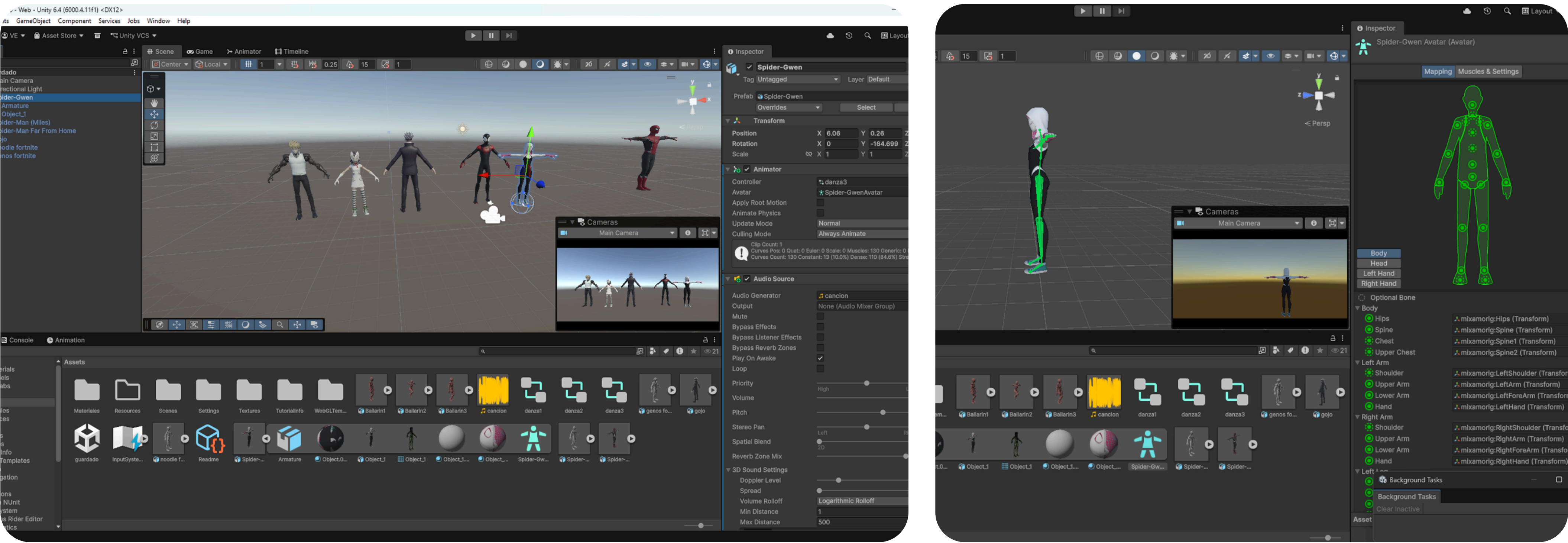
Para el desarrollo de la animación sincronizada se utilizó el motor gráfico Unity, permitiendo crear una escena tridimensional interactiva con múltiples personajes ejecutando coreografías coordinadas al ritmo de una pista musical. Inicialmente se seleccionaron y descargaron modelos tridimensionales junto con animaciones desde la plataforma Mixamo y Sketchfab. Los personajes fueron importados a Unity y configurados mediante el sistema de rigging Humanoid, lo que permitió adaptar correctamente las animaciones al esqueleto de cada modelo y garantizar la compatibilidad entre diferentes personajes.



La escena fue compuesta por seis personajes distribuidos en parejas. La coreografía completa estuvo formada por tres secuencias de baile diferentes, cada una controlada mediante su propio Animator Controller. Estos controladores gestionan la reproducción de las animaciones y permiten organizar de manera estructurada las distintas rutinas de movimiento implementadas en la escena.

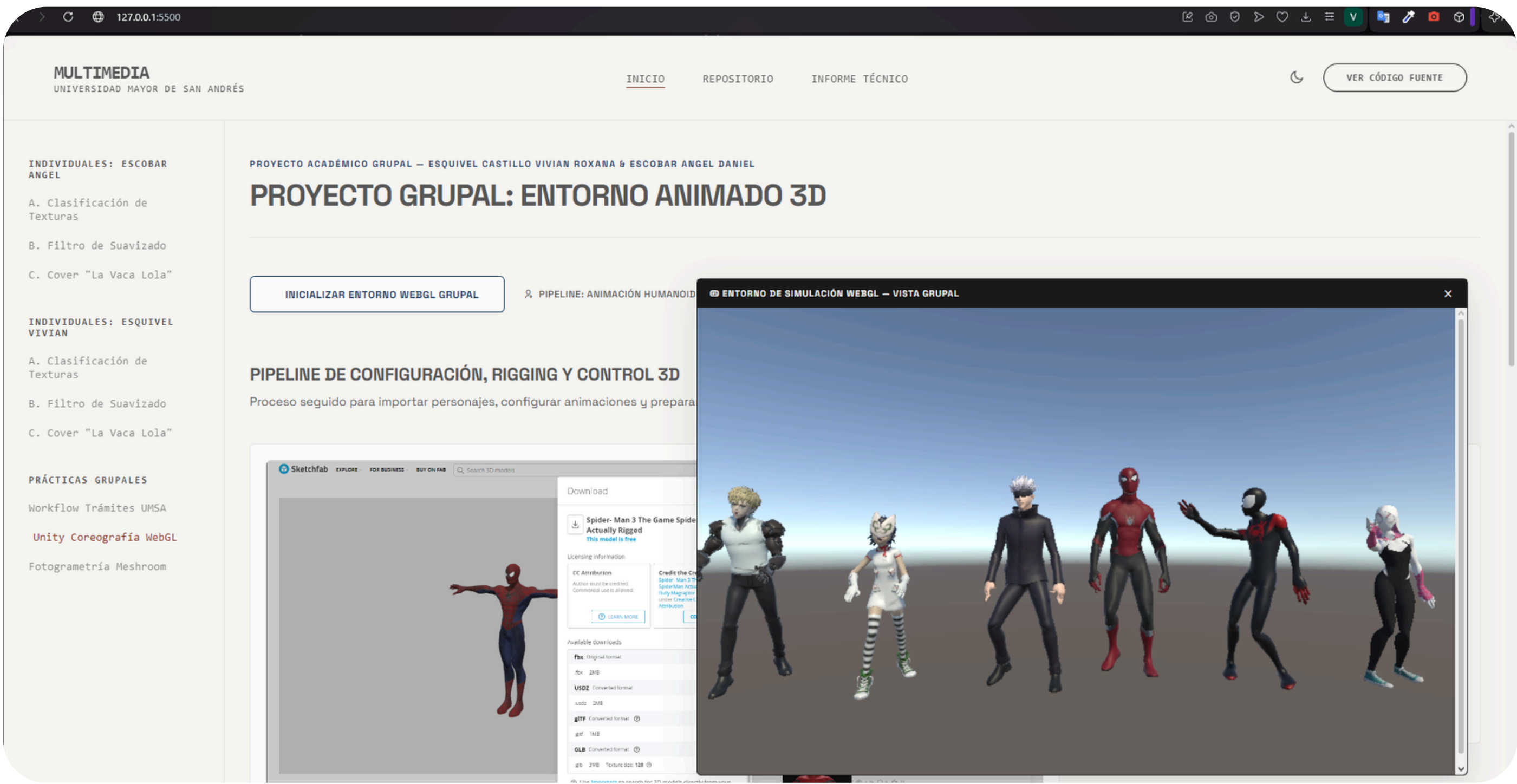
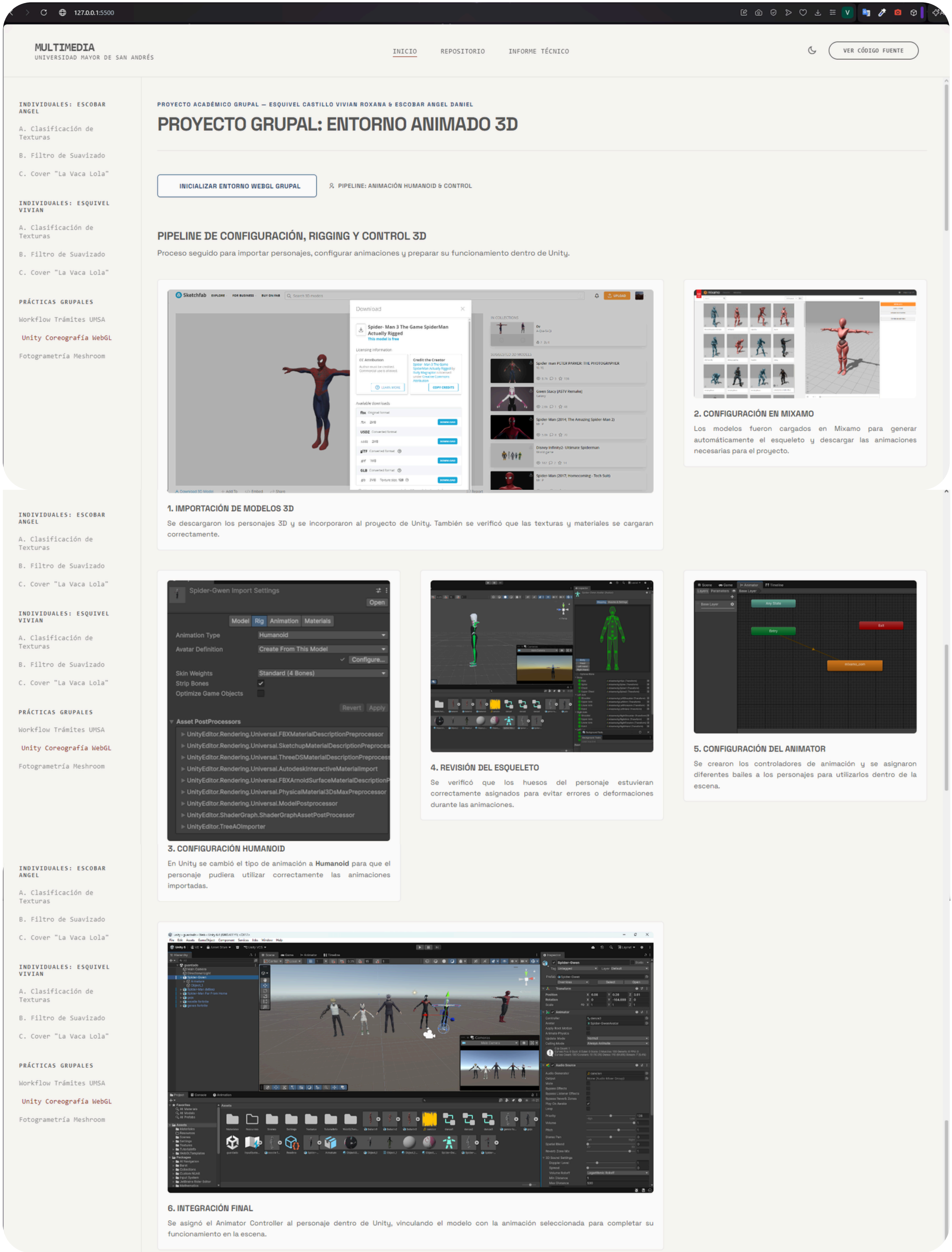


Para lograr la sincronización, las animaciones fueron configuradas de forma que cada pareja de personajes ejecutara simultáneamente los mismos movimientos, generando una presentación coordinada y visualmente uniforme. Asimismo, se incorporó una pista musical de estilo pop que actúa como acompañamiento de la coreografía, buscando mantener la correspondencia temporal entre el ritmo de la música y las secuencias de animación.



Una vez finalizado el desarrollo y las pruebas de funcionamiento, el proyecto fue exportado utilizando la plataforma WebGL de Unity. Para ello se configuraron las opciones correspondientes en Build Settings, ajustando parámetros de resolución y visualización para garantizar la correcta ejecución desde navegadores web modernos sin requerir instalaciones adicionales.

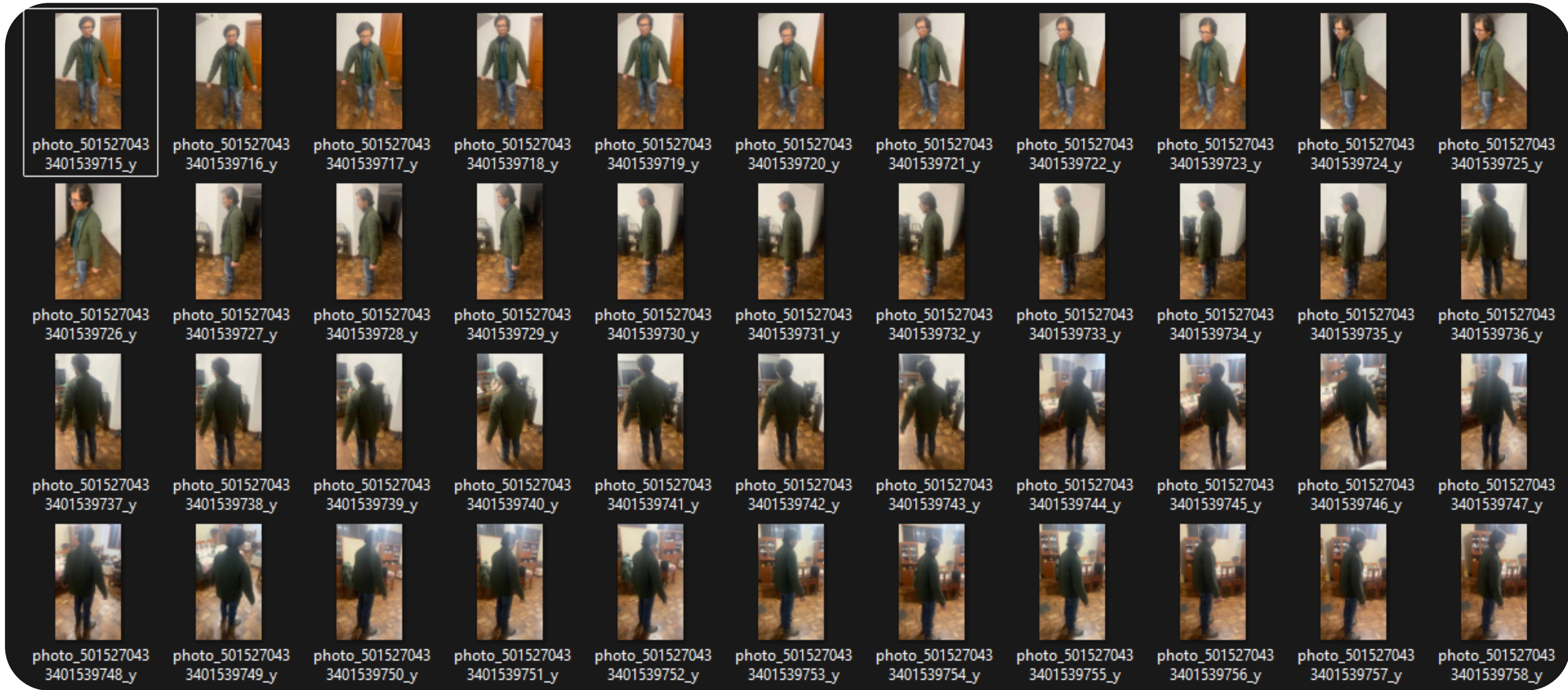
El resultado final fue una aplicación web interactiva capaz de reproducir la coreografía tridimensional sincronizada, cumpliendo con los requisitos establecidos para la actividad.



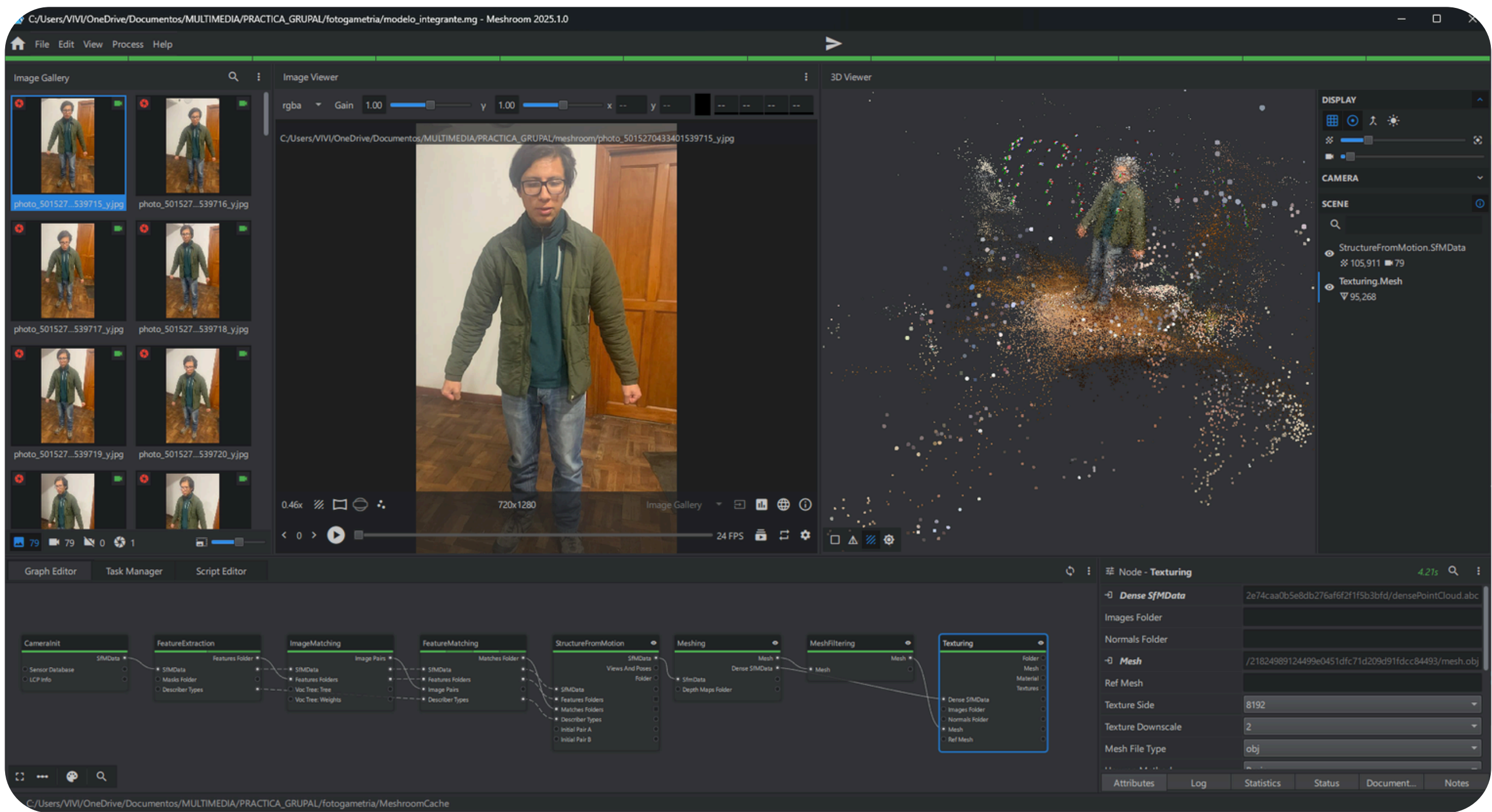
4.2 CONSTRUCCIÓN DE UN AVATAR DIGITAL MEDIANTE FOTOGRAMETRÍA

Para la generación de un avatar digital se empleó la técnica de fotogrametría, la cual permite reconstruir modelos tridimensionales a partir de múltiples fotografías tomadas desde diferentes ángulos.

El proceso comenzó con la captura de aproximadamente 79 fotografías de un integrante del grupo. Las imágenes fueron tomadas alrededor de la persona con el objetivo de cubrir la mayor cantidad posible de detalles y perspectivas, permitiendo una reconstrucción más precisa de la geometría del modelo.



Posteriormente, las fotografías fueron procesadas mediante la herramienta Meshroom, software especializado en reconstrucción tridimensional basado en técnicas de visión por computadora. Durante el procesamiento, la aplicación realizó automáticamente las etapas de detección de características, alineación de cámaras, generación de nube de puntos, construcción de malla y aplicación de texturas.



Una vez finalizada la reconstrucción, el modelo tridimensional generado fue exportad o desde Meshroom utilizando formatos compatibles con aplicaciones web de visualización 3D. Posteriormente, el modelo fue convertido a un formato optimizado para su despliegue en navegadores, permitiendo una mejor integración con tecnologías web modernas.

 texturedMesh.glb	18/6/2026 00:33	Archivo GLB	8.933 KB
 texturedMesh.obj	17/6/2026 23:58	VisualStudio.obj.5...	8.771 KB

Para la visualización se desarrolló un visor tridimensional integrado directamente en la plataforma web del proyecto. Dicho visor permite al usuario interactuar con el modelo generado, observándolo desde diferentes ángulos mediante controles de navegación. Asimismo, se incorporaron funciones de apertura, cierre, desplazamiento y redimensionamiento de la ventana de visualización mediante JavaScript, proporcionando una experiencia interactiva y adaptable.

El resultado obtenido fue un avatar digital tridimensional basado en un integrante real del grupo, visualizable directamente desde la página web sin necesidad de instalar aplicaciones adicionales.

MULTIMEDIA

UNIVERSIDAD MAYOR DE SAN ANDRÉS

INICIO

REPOSITORIO

INFORME TÉCNICO

VER CÓDIGO FUENTE

INDIVIDUALES: ESCOBAR ANGEL

A. Clasificación de Texturas

B. Filtro de Suavizado

C. Cover "La Vaca Lola"

INDIVIDUALES: ESQUIVEL VIVIAN

A. Clasificación de Texturas

B. Filtro de Suavizado

C. Cover "La Vaca Lola"

PRÁCTICAS GRUPALES

Workflow Trámites UMSA

Unity Coreografía WebGL

Fotogrametría Meshroom

INDIVIDUALES: ESCOBAR ANGEL

A. Clasificación de Texturas

B. Filtro de Suavizado

C. Cover "La Vaca Lola"

INDIVIDUALES: ESQUIVEL VIVIAN

A. Clasificación de Texturas

B. Filtro de Suavizado

C. Cover "La Vaca Lola"

PRÁCTICAS GRUPALES

Workflow Trámites UMSA

Unity Coreografía WebGL

Fotogrametría Meshroom

PROYECTO ACADÉMICO GRUPAL — ESQUIVEL CASTILLO VIVIAN ROXANA & ESCOBAR ANGEL DANIEL

RECONSTRUCCIÓN 3D Y FOTOGRAMETRÍA

DESPLEGAR VISOR 3D INTERACTIVO

PIPELINE: MESHRROOM

PIPELINE DE CAPTURA Y PROCESAMIENTO

Proceso seguido para obtener un modelo 3D a partir de fotografías utilizando Meshroom.

1. CAPTURA DE FOTOGRAFÍAS

Se tomaron varias fotografías del integrante desde diferentes ángulos alrededor de la persona. Estas imágenes servirán como base para generar el modelo 3D.

2. CARGA Y PROCESAMIENTO EN MESHRROOM

Las fotografías se importaron en Meshroom. El programa analiza las imágenes, identifica puntos en común y calcula la posición de cada fotografía para reconstruir la escena.

3. GENERACIÓN DE LA RECONSTRUCCIÓN

Meshroom procesa la información obtenida de las fotografías para construir la forma tridimensional del modelo. Debido a las limitaciones de hardware disponibles, algunas etapas avanzadas del proceso no pudieron completarse correctamente.

4. EXPORTACIÓN DEL MODELO

Una vez finalizado el procesamiento, el modelo puede exportarse en formato .obj para ser utilizado posteriormente en otros programas de modelado o motores gráficos.

MULTIMEDIA

UNIVERSIDAD MAYOR DE SAN ANDRÉS

INICIO

REPOSITORIO

INFORME TÉCNICO

VER CÓDIGO FUENTE

INDIVIDUALES: ESCOBAR ANGEL

A. Clasificación de Texturas

B. Filtro de Suavizado

C. Cover "La Vaca Lola"

INDIVIDUALES: ESQUIVEL VIVIAN

A. Clasificación de Texturas

B. Filtro de Suavizado

C. Cover "La Vaca Lola"

PRÁCTICAS GRUPALES

Workflow Trámites UMSA

Unity Coreografía WebGL

Fotogrametría Meshroom

INDIVIDUALES: ESCOBAR ANGEL

A. Clasificación de Texturas

B. Filtro de Suavizado

C. Cover "La Vaca Lola"

INDIVIDUALES: ESQUIVEL VIVIAN

A. Clasificación de Texturas

B. Filtro de Suavizado

C. Cover "La Vaca Lola"

PRÁCTICAS GRUPALES

Workflow Trámites UMSA

Unity Coreografía WebGL

Fotogrametría Meshroom

PROYECTO ACADÉMICO GRUPAL — ESQUIVEL CASTILLO VIVIAN ROXANA & ESCOBAR ANGEL DANIEL

RECONSTRUCCIÓN 3D Y FOTOGRAMETRÍA

DESPLEGAR VISOR 3D INTERACTIVO

PIPELINE: MESHRROOM

PIPELINE DE CAPTURA Y PROCESAMIENTO

Proceso seguido para obtener un modelo 3D a partir de fotografías utilizando Meshroom.

1. CAPTURA DE FOTOGRAFÍAS

Se tomaron varias fotografías del integrante desde diferentes

2. CARGA Y PROCESAMIENTO EN MESHRROOM

Las fotografías se importaron en Meshroom. El programa analiza las imágenes, identifica puntos en común y calcula la posición de cada fotografía para reconstruir la escena.

MANIPULACIÓN DE MODELO 3D

**UNIVERSIDAD MAYOR DE SAN ANDRÉS
CARRERA DE INFORMÁTICA
FACULTAD DE CIENCIAS PURAS Y NATURALES**



**INF-263
DESARROLLO DE APLICACIONES MULTIMEDIA**

**“PROYECTO INDIVIDUAL: Implementación de Algoritmos de
Procesamiento Digital de Imágenes y Contenido Multimedia”**

ESTUDIANTE:

ESQUIVEL CASTILLO VIVIAN ROXANA

DOCENTE:

Ph.D. MOISES MARTIN SILVA CHOQUE

LA PAZ -BOLIVIA
2026

1. INTRODUCCIÓN

El procesamiento digital de imágenes y la producción multimedia constituyen áreas fundamentales dentro del desarrollo de aplicaciones interactivas modernas. Estas disciplinas permiten analizar información visual, transformar contenidos digitales y generar experiencias audiovisuales que integran imágenes, audio, animación y elementos tridimensionales.

El presente trabajo reúne tres actividades orientadas a la aplicación práctica de conceptos relacionados con el procesamiento de imágenes y la creación multimedia. La primera actividad consistió en el desarrollo de un sistema de clasificación de superficies mediante el análisis de características cromáticas obtenidas directamente de los píxeles de una imagen. La segunda actividad abordó la implementación de un filtro de suavizado basado en ventanas de análisis de 3×3 píxeles para mejorar la calidad visual de imágenes digitales. Finalmente, la tercera actividad comprendió la elaboración de una producción multimedia basada en la canción infantil “La Vaca Lola”, integrando procesamiento de audio, síntesis de voz asistida por inteligencia artificial y animación tridimensional desarrollada en Unity.

Mediante la realización de estas actividades fue posible aplicar conocimientos relacionados con programación, manipulación de imágenes, procesamiento de audio, gráficos tridimensionales y herramientas multimedia, demostrando la integración de diferentes tecnologías dentro de proyectos digitales interactivos.

2. METODOLOGÍA UTILIZADA

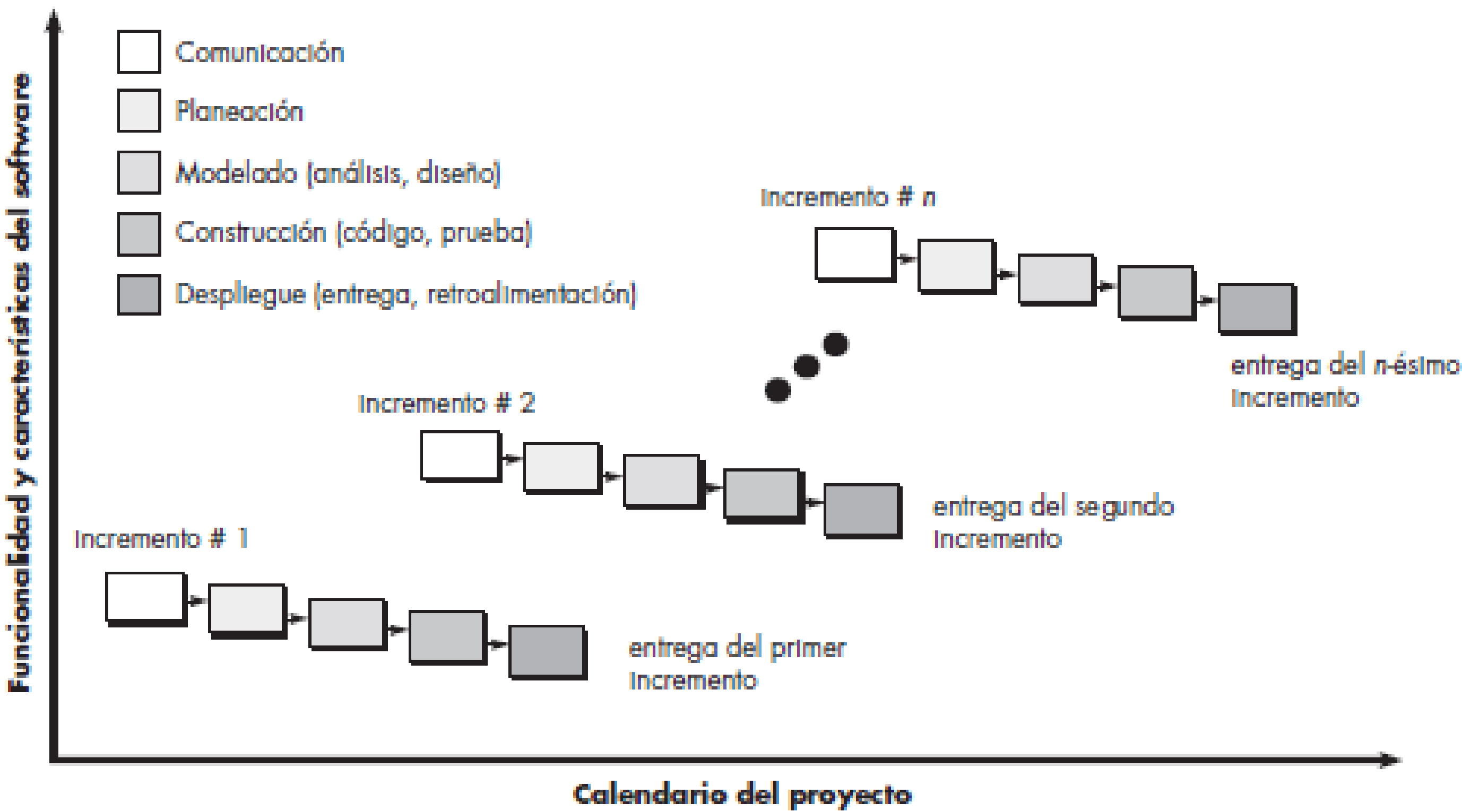
El desarrollo de las actividades se realizó siguiendo una metodología incremental basada en el análisis, diseño, implementación y validación de cada solución propuesta. Inicialmente se estudiaron los requerimientos específicos de cada ejercicio con el propósito de identificar los algoritmos, herramientas y recursos necesarios para su desarrollo.

En la actividad de clasificación de texturas se implementó una estrategia basada en el análisis de regiones de píxeles y la comparación de características cromáticas almacenadas en una base de datos. Para ello se desarrolló una aplicación de escritorio capaz de registrar muestras de referencia y realizar procesos de reconocimiento sobre nuevas imágenes.

Para la implementación del filtro de suavizado se trabajó directamente a nivel de píxel utilizando imágenes Bitmap. Se aplicaron técnicas de detección de bordes y filtrado espacial mediante ventanas de análisis de 3×3 píxeles, permitiendo observar los efectos producidos por el procesamiento digital sobre la imagen original.

En la producción multimedia se utilizaron herramientas especializadas para la edición y transformación de audio, generación de voz mediante inteligencia artificial y desarrollo de animaciones tridimensionales. El contenido audiovisual fue integrado dentro de Unity utilizando personajes, animaciones y recursos tridimensionales obtenidos de diferentes plataformas especializadas.

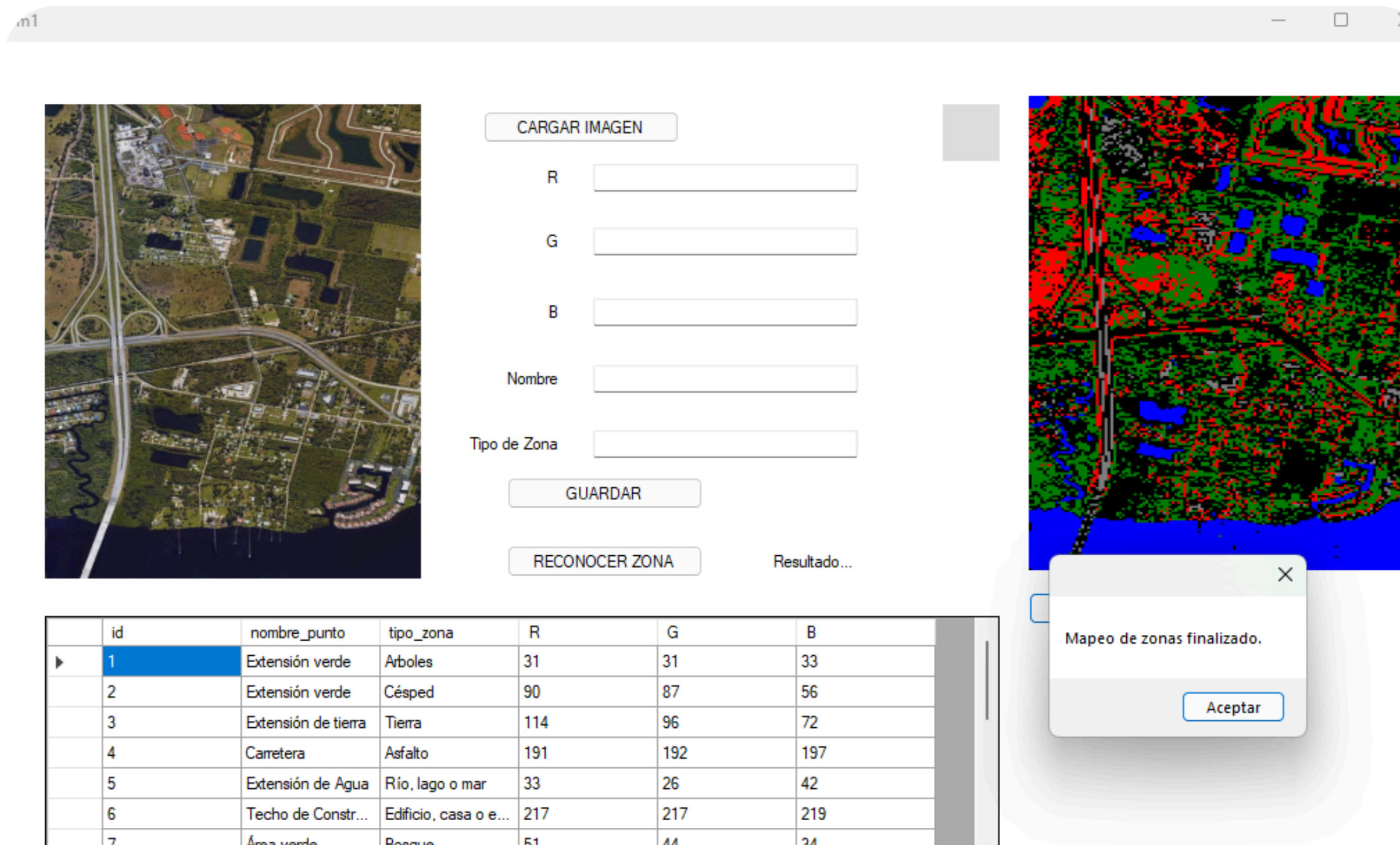
Finalmente, en todas las actividades se realizaron pruebas funcionales y verificaciones visuales con el fin de evaluar el comportamiento de los algoritmos implementados y validar los resultados obtenidos respecto a los objetivos planteados.



4. ACTIVIDAD 1: CLASIFICACIÓN DE TEXTURAS

4.1 DESCRIPCIÓN GENERAL

La actividad consistió en desarrollar una aplicación capaz de identificar diferentes tipos de superficies presentes en una imagen digital mediante el análisis directo de los píxeles. El sistema fue implementado utilizando C# en Visual Studio bajo la plataforma Windows Forms y empleó SQL Server para almacenar muestras de referencia asociadas a distintas categorías de textura.



4.2 METODOLOGÍA DE IMPLEMENTACIÓN

El proceso comienza con la carga de una imagen por parte del usuario. Una vez visualizada, el usuario selecciona un punto específico mediante un clic sobre la imagen. A partir de dicha posición, el sistema analiza una región cuadrada de 5×5 píxeles alrededor del punto seleccionado.

Para representar la textura de la zona, se calcula el promedio de los valores de los canales Rojo (R), Verde (G) y Azul (B) de todos los píxeles contenidos en la ventana de análisis. El resultado corresponde a un color representativo de la superficie seleccionada.

Posteriormente, el usuario puede registrar la muestra indicando un nombre descriptivo y el tipo de zona al que pertenece. Los valores promedio de color son almacenados en una base de datos SQL Server, permitiendo construir un conjunto de referencias para futuras comparaciones.

Durante la etapa de reconocimiento, el sistema vuelve a calcular el color promedio de la región seleccionada y compara sus componentes RGB con todas las muestras almacenadas en la base de datos. Para ello se utiliza una medida de distancia basada en la diferencia absoluta entre los valores de los canales de color. La textura cuya distancia sea menor respecto a la muestra analizada es considerada la coincidencia más cercana y se presenta como resultado del reconocimiento.

```
private void btnCargar_Click(object sender, EventArgs e)
{
    openFileDialog1.Filter = "Archivos JPG|*.jpg| Archivos PNG|*.png| Archivos JPEG|*.jpeg";
    if (openFileDialog1.ShowDialog() == DialogResult.OK)
    {
        bmp = new Bitmap(openFileDialog1.FileName);
        pictureBox1.Image = bmp;
    }
}

private void btnGuardar_Click(object sender, EventArgs e)
{
    using (SqlConnection con = new SqlConnection("server=(local); user=multimedia; pwd=1234; database=bd_Texturas"))
    {
        string query = "INSERT INTO reconocimiento_puntos(nombre_punto, tipo_zona, R, G, B) VALUES (@nombre, @zona, @r, @g, @b)";
        SqlCommand cmd = new SqlCommand(query, con);

        cmd.Parameters.AddWithValue("@nombre", txtNombre.Text);
        cmd.Parameters.AddWithValue("@zona", txtTipoZona.Text);
        cmd.Parameters.AddWithValue("@r", int.Parse(txtR.Text));
        cmd.Parameters.AddWithValue("@g", int.Parse(txtG.Text));
        cmd.Parameters.AddWithValue("@b", int.Parse(txtB.Text));

        con.Open();
        cmd.ExecuteNonQuery();
    }

    RefrescarGrid();

    pctColor.BackColor = SystemColors.Control;
    txtR.Clear(); txtG.Clear(); txtB.Clear();
    txtNombre.Clear(); txtTipoZona.Clear();
}

private void btnReconocer_Click(object sender, EventArgs e)
{
    if (bmp == null)
    {
        MessageBox.Show("Carga una imagen primero");
        return;
    }

    if (lastX < 0 || lastY < 0)
    {
        MessageBox.Show("Haz clic en la imagen primero");
        return;
    }

    // El reconocimiento también se basa en la textura promedio del punto
    // cliqueado
    Color c = ObtenerPromedioTextura(bmp, lastX, lastY, TAMANO_TEXTURA);

    using (SqlConnection con = new SqlConnection("server=(local); user=multimedia; pwd=1234; database=bd_Texturas"))
    {
        con.Open();
        SqlCommand cmd = new SqlCommand("SELECT * FROM reconocimiento_puntos", con);
        SqlDataReader dr = cmd.ExecuteReader();

        string zona = "No identificada";
        int mejor = int.MaxValue;

        while (dr.Read())
        {
            int r = Convert.ToInt32(dr["R"]);
            int g = Convert.ToInt32(dr["G"]);
            int b = Convert.ToInt32(dr["B"]);

            int dist = Math.Abs(c.R - r) +
                Math.Abs(c.G - g) +
                Math.Abs(c.B - b);

            if (dist < mejor)
            {
                mejor = dist;
                zona = dr["tipo_zona"].ToString();
            }
        }
        dr.Close();
        lblResultado.Text = zona;
        MessageBox.Show("Zona detectada: " + zona);
    }
}
```

4.3 TECNOLOGÍAS UTILIZADAS

Para el desarrollo de la aplicación se utilizaron las siguientes herramientas:

- Visual Studio como entorno de desarrollo.
- Lenguaje de programación C#.
- Windows Forms para la construcción de la interfaz gráfica.
- SQL Server para el almacenamiento de muestras de referencia.
- Biblioteca System.Drawing para la manipulación y análisis de imágenes.

4.4 RESULTADOS OBTENIDOS

La aplicación desarrollada permitió registrar diferentes muestras de superficies y realizar procesos de identificación basados en características cromáticas obtenidas directamente de los píxeles de la imagen. El uso de ventanas de análisis de 5×5 píxeles permitió reducir pequeñas variaciones de color presentes en imágenes reales, mejorando la estabilidad del reconocimiento.

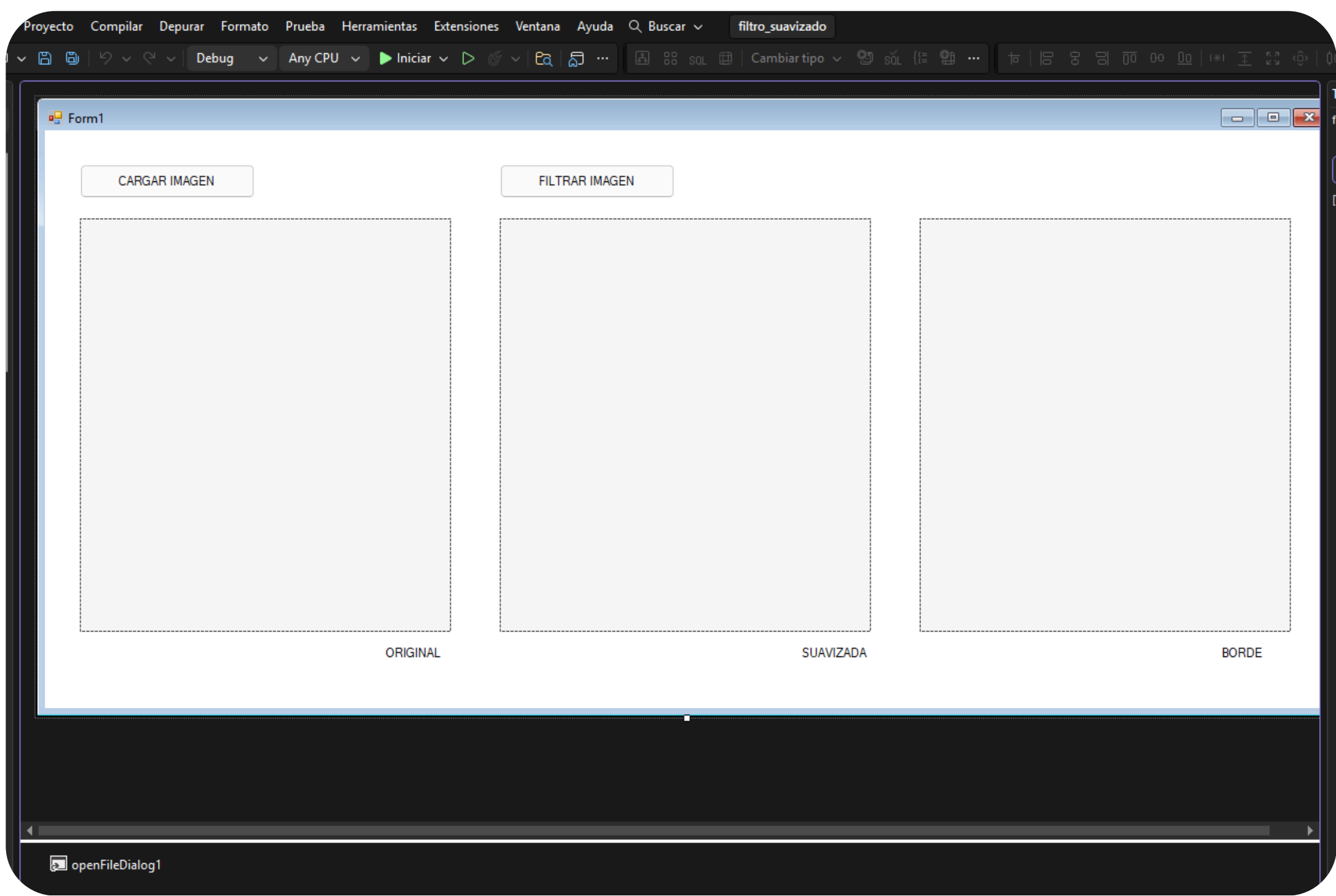
Los resultados demostraron que es posible diferenciar zonas con características visuales distintas mediante el análisis de información RGB promedio, constituyendo una aproximación básica a los sistemas de clasificación de superficies utilizados en aplicaciones de visión artificial y procesamiento digital de imágenes.

[illegible]

5. ACTIVIDAD 2: IMPLEMENTACIÓN DE UN FILTRO DE SUAVIZADO

5.1 DESCRIPCIÓN GENERAL

La actividad consistió en desarrollar una aplicación capaz de reducir variaciones bruscas presentes en una imagen digital mediante técnicas de procesamiento a nivel de píxel. La implementación fue realizada en C# utilizando Visual Studio y Windows Forms, permitiendo cargar imágenes, aplicar el procesamiento y visualizar los resultados obtenidos.



5.2 METODOLOGÍA DE IMPLEMENTACIÓN

El sistema desarrollado trabaja directamente sobre los valores RGB de cada píxel de la imagen. Inicialmente, el usuario selecciona una imagen que es cargada en memoria mediante un objeto Bitmap.

Antes de aplicar el suavizado, se realiza una etapa de detección de bordes utilizando el operador de Laplace. Para ello se emplea una máscara de convolución de tamaño 3x3 que permite identificar cambios bruscos de intensidad entre píxeles vecinos. La respuesta obtenida es utilizada para calcular una magnitud de borde asociada a cada posición de la imagen.

Posteriormente, el sistema evalúa un umbral predefinido. Cuando la magnitud calculada supera dicho valor, la región es considerada un borde y se aplica un filtro de promedio utilizando una ventana de 3x3 píxeles. El filtro calcula el promedio de los nueve píxeles vecinos para cada componente de color (Rojo, Verde y Azul), generando una transición más suave entre intensidades.

En aquellas zonas donde no se detectan bordes significativos, el sistema conserva el valor original del píxel, evitando modificaciones innecesarias sobre regiones homogéneas de la imagen.

5.3 TECNOLOGÍAS UTILIZADAS

Las herramientas empleadas para el desarrollo fueron:

- Visual Studio como entorno de programación.
- Lenguaje C#.
- Windows Forms para la interfaz gráfica.
- Biblioteca System.Drawing para la manipulación de imágenes Bitmap.
- Algoritmos de convolución para detección de bordes y filtrado espacial.

```
Bitmap bmpOriginal;
Bitmap bmpResultado;
Bitmap bmpBordes;
public Form1()
{
    InitializeComponent();
}

private void openFileDialog1_FileOk(object sender, CancelEventArgs e)
{
}

private void btnCargar_Click(object sender, EventArgs e)
{
    openFileDialog1.Filter = "Archivos JPG|*.jpg| Archivos PNG|*.png| Archivos JPEG|*.jpeg";
    openFileDialog1.InitialDirectory = Environment.GetFolderPath(Environment.SpecialFolder.Desktop);

    if (openFileDialog1.ShowDialog() == DialogResult.OK)
    {
        bmpOriginal = new Bitmap(openFileDialog1.FileName);
        picOriginal.Image = bmpOriginal;
    }
}

private void btnFiltrar_Click(object sender, EventArgs e)
{
    if (bmpOriginal == null)
    {
        MessageBox.Show("Carga una imagen primero");
        return;
    }

    int anchoReal = bmpOriginal.Width;
    int altoReal = bmpOriginal.Height;

    bmpResultado = new Bitmap(anchoReal, altoReal);

    int[, ] kernelLaplace = new int[3, 3] {
        { 0, 1, 0 },
        { 1, -4, 1 },
        { 0, 1, 0 }
    };

    int umbralBorde = 25;

    for (int x = 0; x < anchoReal; x++)
    {
        for (int y = 0; y < altoReal; y++)
        {
            if (x == 0 || y == 0 || x == anchoReal - 1 || y == altoReal - 1)
            {
                bmpResultado.SetPixel(x, y, bmpOriginal.GetPixel(x, y));
            }
        }
    }

    for (int x = 1; x < anchoReal - 1; x++)
    {
        for (int y = 1; y < altoReal - 1; y++)
        {
            // 1. PASO: DETECCIÓN DE BORDE CON LAPLACE
            int laplaceR = 0, laplaceG = 0, laplaceB = 0;

            for (int kx = -1; kx <= 1; kx++)
            {
                for (int ky = -1; ky <= 1; ky++)
                {
                    Color pixelVecino = bmpOriginal.GetPixel(x + kx, y + ky);
                    int peso = kernelLaplace[kx + 1, ky + 1];

                    laplaceR += pixelVecino.R * peso;
                    laplaceG += pixelVecino.G * peso;
                    laplaceB += pixelVecino.B * peso;
                }
            }

            int magnitudBorde = (Math.Abs(laplaceR) + Math.Abs(laplaceG) + Math.Abs(laplaceB)) / 3;

            // 2. PASO: CONDICIONAL SEGÚN TU LICENCIADO
            if (magnitudBorde > umbralBorde)
            {
                // SI ES UN BORDE -> Aplicamos el suavizado
                int sumaR = 0, sumaG = 0, sumaB = 0;

                for (int kx = -1; kx <= 1; kx++)
                {
                    for (int ky = -1; ky <= 1; ky++)
                    {
                        Color pixelVecino = bmpOriginal.GetPixel(x + kx, y + ky);
                        sumaR += pixelVecino.R;
                        sumaG += pixelVecino.G;
                        sumaB += pixelVecino.B;
                    }
                }

                int promR = sumaR / 9;
                int promG = sumaG / 9;
                int promB = sumaB / 9;

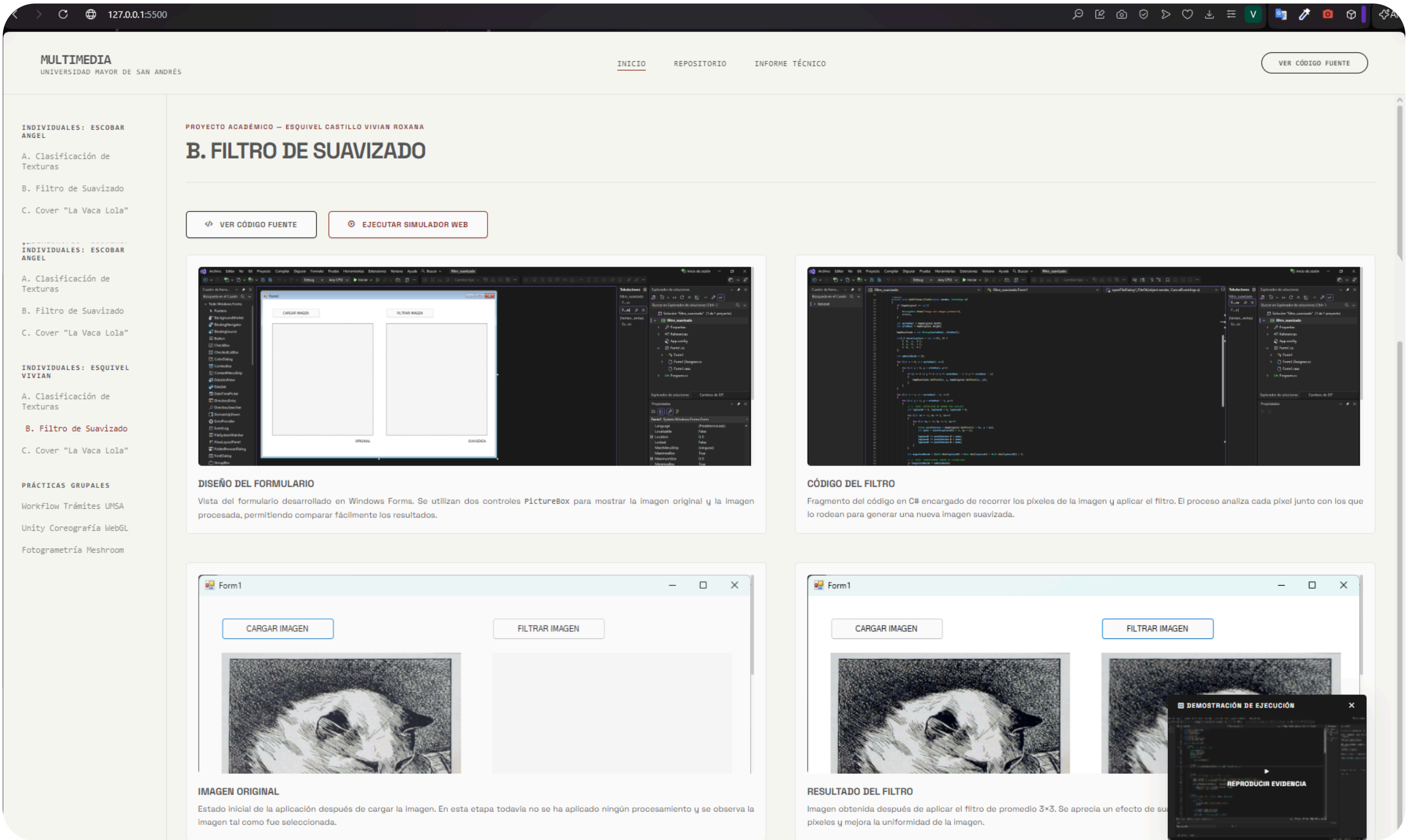
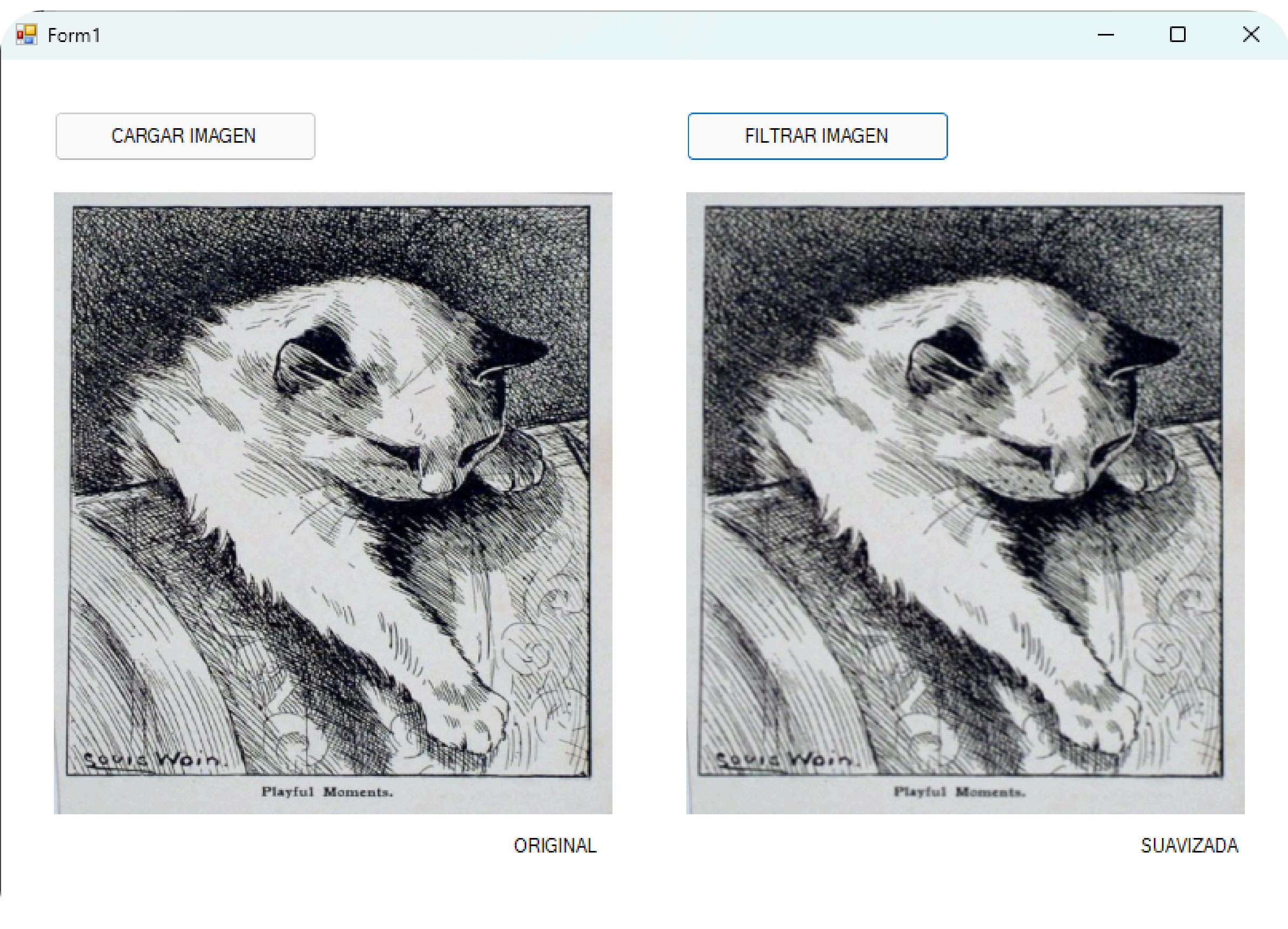
                bmpResultado.SetPixel(x, y, Color.FromArgb(promR, promG, promB));
            }
            else
            {
                // NO ES UN BORDE -> Se queda el color original de la foto sin alterar
                bmpResultado.SetPixel(x, y, bmpOriginal.GetPixel(x, y));
            }
        }
    }

    picResultado.Image = bmpResultado;
    MessageBox.Show("Filtro de suavizado por Laplace aplicado con éxito");
}
```

5.4 RESULTADOS OBTENIDOS

La aplicación permitió aplicar un proceso de suavizado selectivo sobre imágenes digitales mediante ventanas de análisis de 3×3 píxeles. La incorporación previa del operador de Laplace permitió identificar regiones con cambios importantes de intensidad y aplicar el filtrado únicamente en dichas zonas.

Como resultado, se obtuvo una reducción de irregularidades visuales y transiciones más suaves en las áreas procesadas, manteniendo al mismo tiempo gran parte de los detalles presentes en regiones uniformes. La comparación visual entre la imagen original y la imagen procesada evidenció el efecto del filtrado implementado y su utilidad dentro del procesamiento digital de imágenes.



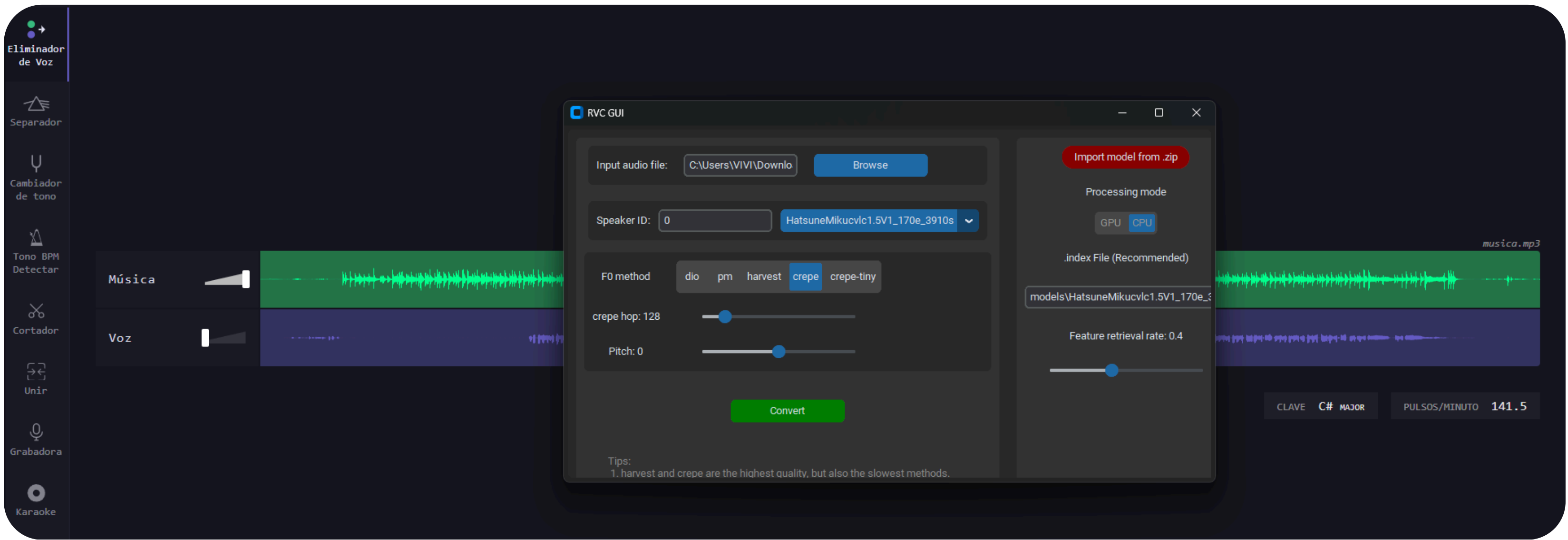
6. ACTIVIDAD 3: PRODUCCIÓN MULTIMEDIA - COVER DE “LA VACA LOLA”

6.1 DESCRIPCIÓN GENERAL

La actividad consistió en desarrollar una producción multimedia basada en la canción infantil “La Vaca Lola”, adaptándola mediante técnicas de procesamiento de audio, síntesis de voz asistida por inteligencia artificial y animación tridimensional en Unity. El objetivo fue generar una versión personalizada de la canción acompañada por una representación visual sincronizada con el contenido sonoro.

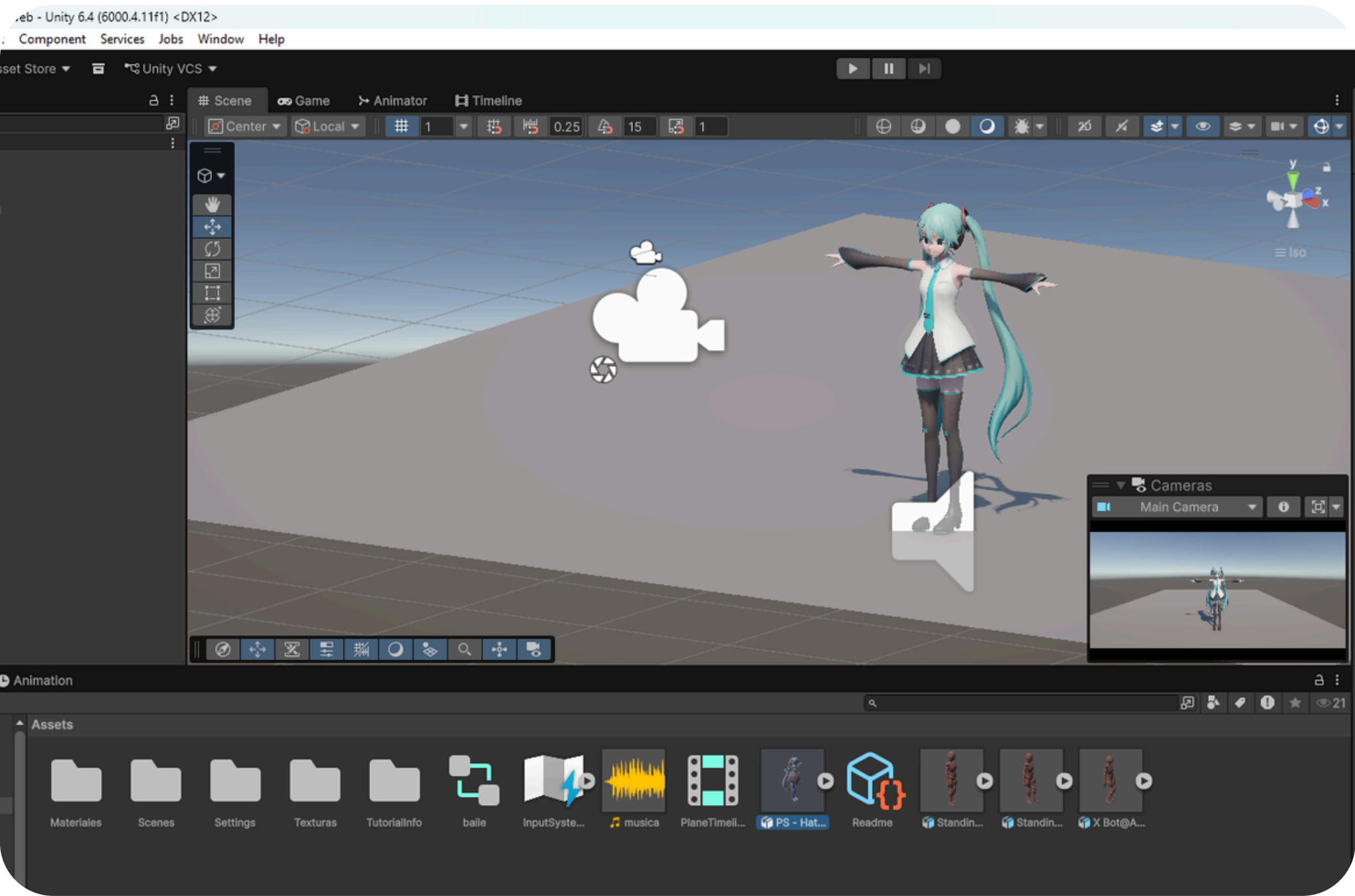
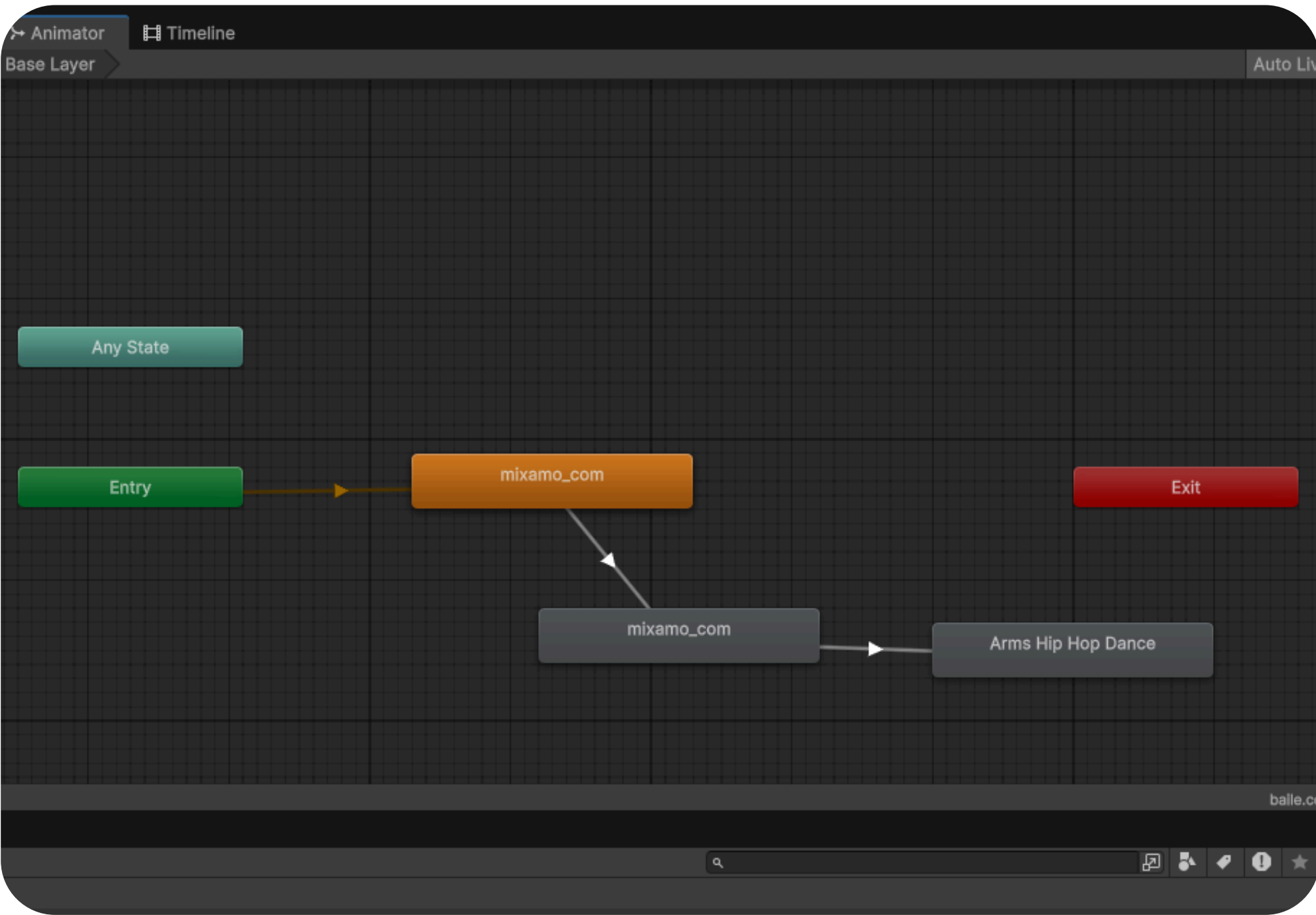
6.2 METODOLOGÍA DE IMPLEMENTACIÓN

El proceso comenzó con la obtención de la canción original y la separación de sus componentes de audio. Mediante herramientas de procesamiento sonoro se extrajo la pista vocal de la instrumental, permitiendo trabajar de forma independiente con la voz y la música de fondo. Posteriormente, la pista vocal fue procesada mediante herramientas de conversión de voz basadas en inteligencia artificial. Para ello se utilizó un modelo de clonación y transformación de voz que permitió reemplazar la voz original por una nueva interpretación digital, manteniendo el contenido y la estructura de la canción.



Una vez generado el nuevo audio, se procedió al desarrollo de la animación en Unity. Se empleó un personaje tridimensional configurado mediante el sistema Humanoid y se incorporaron animaciones obtenidas desde Mixamo. La secuencia final estuvo compuesta por tres animaciones consecutivas organizadas mediante un Animator Controller, permitiendo construir una coreografía continua durante toda la reproducción musical.

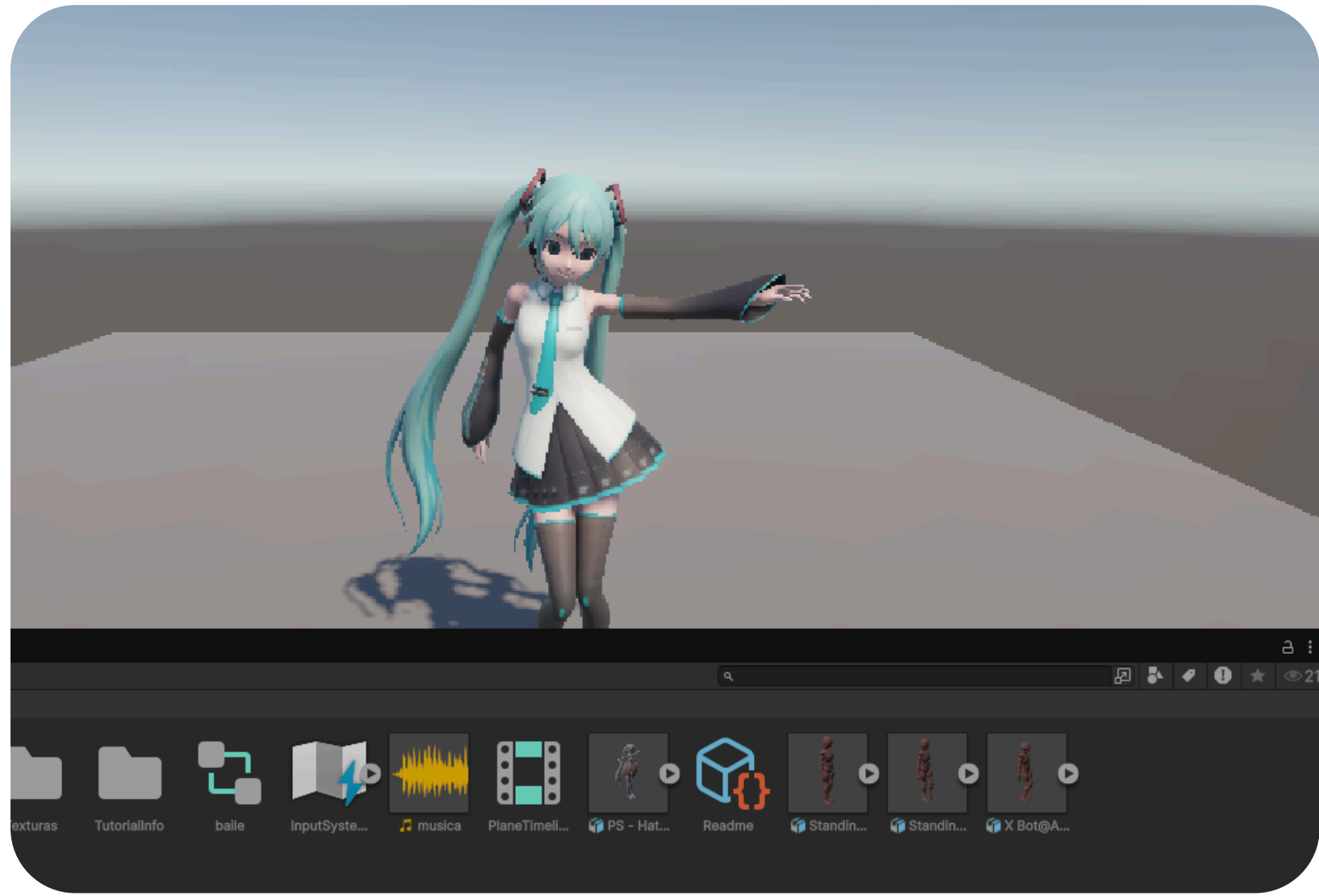
La sincronización entre audio y animación fue realizada ajustando la duración y transición de las secuencias de movimiento para mantener correspondencia con el ritmo de la canción. De esta manera, el personaje ejecuta diferentes movimientos de baile acompañando el desarrollo de la pista musical generada.



6.3 HERRAMIENTAS UTILIZADAS

Las principales herramientas utilizadas durante el desarrollo fueron:

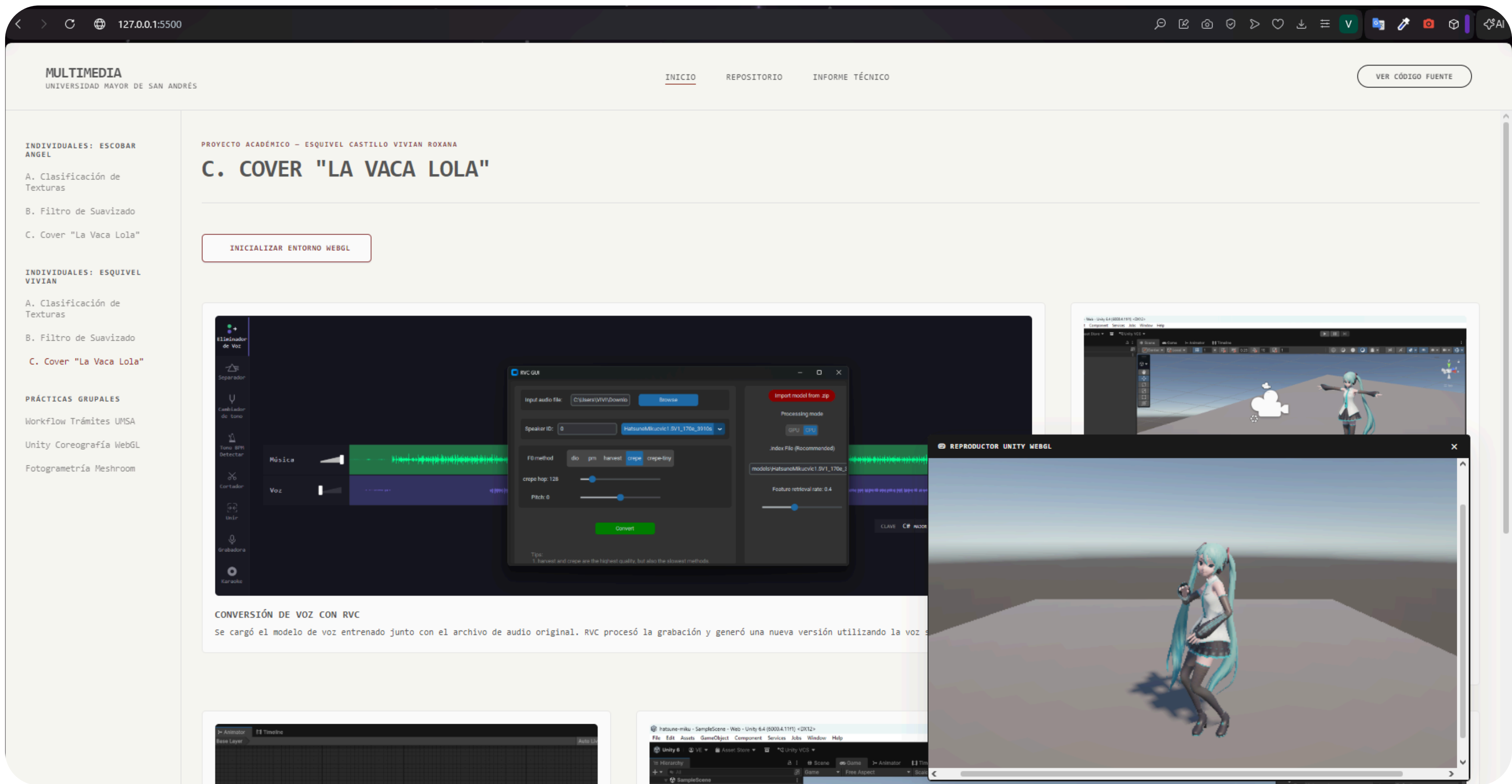
- • Unity para la creación de la escena tridimensional.
- • Mixamo para la obtención de animaciones y configuración del personaje.
- • Audacity para el procesamiento y edición de audio.
- • Vocal Remover para la separación de pistas vocales e instrumentales.
- • Modelos de inteligencia artificial para conversión y síntesis de voz.
- • Sketchfab como fuente de recursos tridimensionales complementarios.



6.4 RESULTADOS OBTENIDOS

Como resultado se obtuvo una producción multimedia interactiva que combina procesamiento de audio, inteligencia artificial y animación tridimensional. La canción fue reinterpretada mediante una nueva voz generada digitalmente y acompañada por una secuencia animada sincronizada con la música.

La integración de múltiples herramientas permitió desarrollar una experiencia audiovisual completa, demostrando la aplicación práctica de tecnologías de edición de audio, síntesis de voz y gráficos tridimensionales dentro de un mismo proyecto multimedia.



7. CONCLUSIONES

Las actividades desarrolladas permitieron aplicar de manera práctica conceptos fundamentales del procesamiento digital de imágenes y la producción multimedia. A través de la implementación de algoritmos de análisis y transformación de imágenes fue posible comprender el funcionamiento de técnicas utilizadas en sistemas de reconocimiento visual y mejora de imágenes digitales.

La clasificación de superficies demostró la utilidad del análisis de características cromáticas para la identificación de diferentes zonas dentro de una imagen, mientras que la implementación del filtro de suavizado permitió observar los efectos del procesamiento espacial sobre la calidad visual de los datos digitales.

Por otra parte, la producción multimedia evidenció la integración de diversas tecnologías relacionadas con audio, inteligencia artificial, animación tridimensional y motores gráficos modernos. La combinación de estas herramientas permitió generar contenido audiovisual interactivo y ampliar la comprensión de los procesos involucrados en el desarrollo multimedia actual.

En conjunto, las actividades contribuyeron al fortalecimiento de habilidades técnicas en programación, procesamiento digital de imágenes, modelado multimedia y utilización de herramientas especializadas para la creación de soluciones interactivas.

**UNIVERSIDAD MAYOR DE SAN ANDRÉS
CARRERA DE INFORMÁTICA
FACULTAD DE CIENCIAS PURAS Y NATURALES**



**INF-263
DESARROLLO DE APLICACIONES MULTIMEDIA**

**“PROYECTO INDIVIDUAL: Implementación de Algoritmos de
Procesamiento Digital de Imágenes y Producción Multimedia”**

ESTUDIANTE:

ESCOBAR ANGEL DANIEL

DOCENTE:

Ph.D. MOISES MARTIN SILVA CHOQUE

LA PAZ -BOLIVIA
2026

1. INTRODUCCIÓN

El procesamiento digital de imágenes y las tecnologías multimedia constituyen herramientas fundamentales dentro del desarrollo de aplicaciones interactivas modernas. Estas áreas permiten analizar información visual, transformar contenidos digitales y crear experiencias audiovisuales mediante la integración de imágenes, audio, animación y gráficos tridimensionales.

El presente trabajo reúne tres actividades orientadas a la aplicación práctica de conceptos relacionados con el procesamiento de imágenes y la producción multimedia. La primera actividad consistió en el desarrollo de un sistema de reconocimiento de texturas mediante el análisis de información cromática obtenida directamente de imágenes digitales. La segunda actividad abordó la implementación de un filtro de suavizado basado en detección de bordes y procesamiento espacial de píxeles. Finalmente, la tercera actividad comprendió la creación de un cover de la canción “La Vaca Lola”, integrando técnicas de procesamiento de audio, conversión de voz mediante inteligencia artificial y animación tridimensional desarrollada en Unity.

La realización de estas actividades permitió fortalecer conocimientos relacionados con programación, procesamiento digital de imágenes, inteligencia artificial aplicada al audio y producción multimedia interactiva.

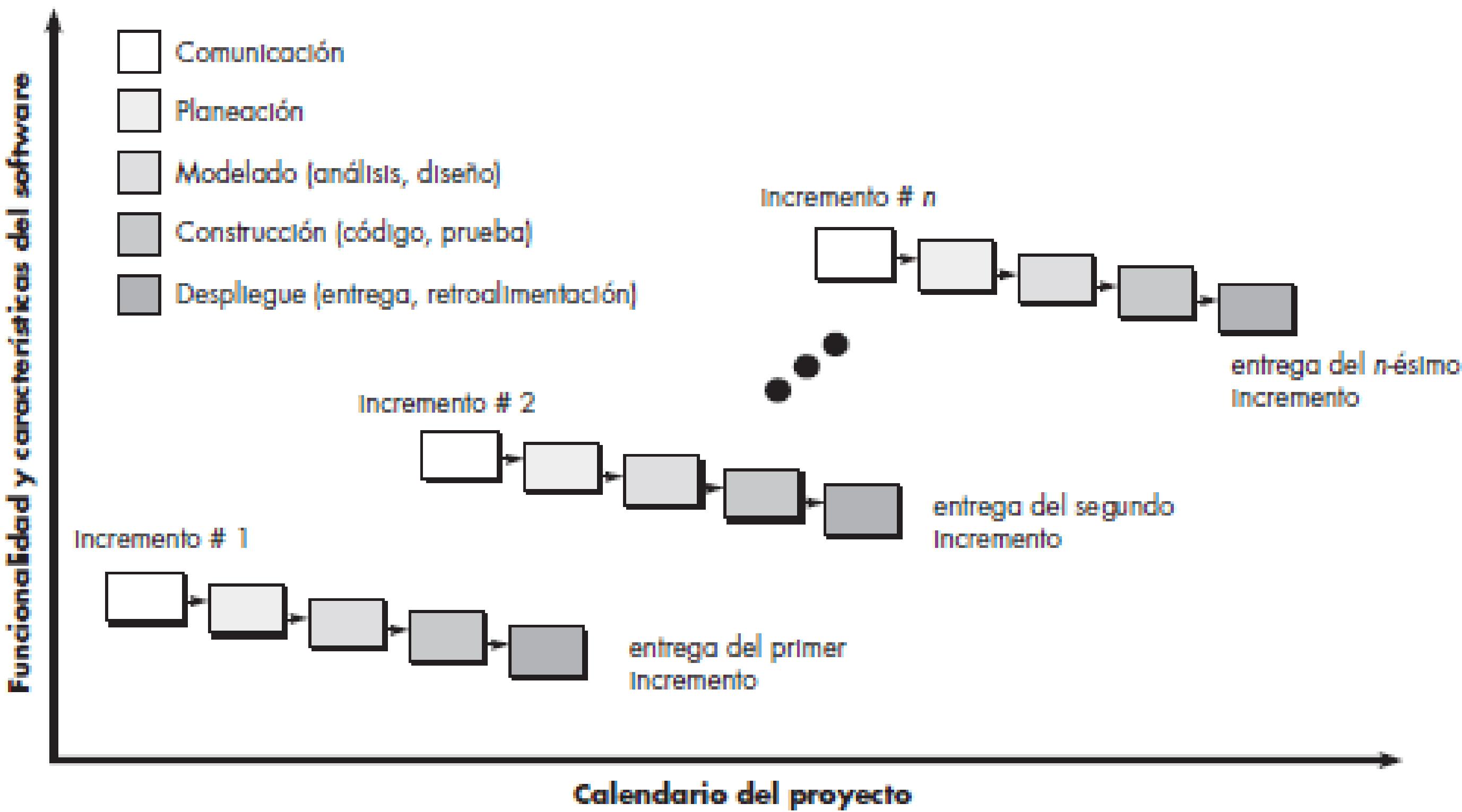
2. METODOLOGÍA UTILIZADA

El desarrollo de las actividades se realizó mediante una metodología incremental basada en las etapas de análisis, diseño, implementación y validación. Inicialmente se identificaron los requerimientos específicos de cada práctica, definiendo los algoritmos, herramientas y recursos necesarios para su construcción.

En las actividades relacionadas con procesamiento de imágenes se trabajó directamente sobre la información contenida en los píxeles de las imágenes digitales, desarrollando algoritmos capaces de analizar características cromáticas y aplicar transformaciones mediante filtros espaciales.

Para la actividad multimedia se emplearon herramientas especializadas para edición de audio, conversión de voz mediante modelos de inteligencia artificial y desarrollo de entornos tridimensionales utilizando Unity. Los diferentes recursos fueron integrados dentro de una misma aplicación con el propósito de generar una experiencia audiovisual completa.

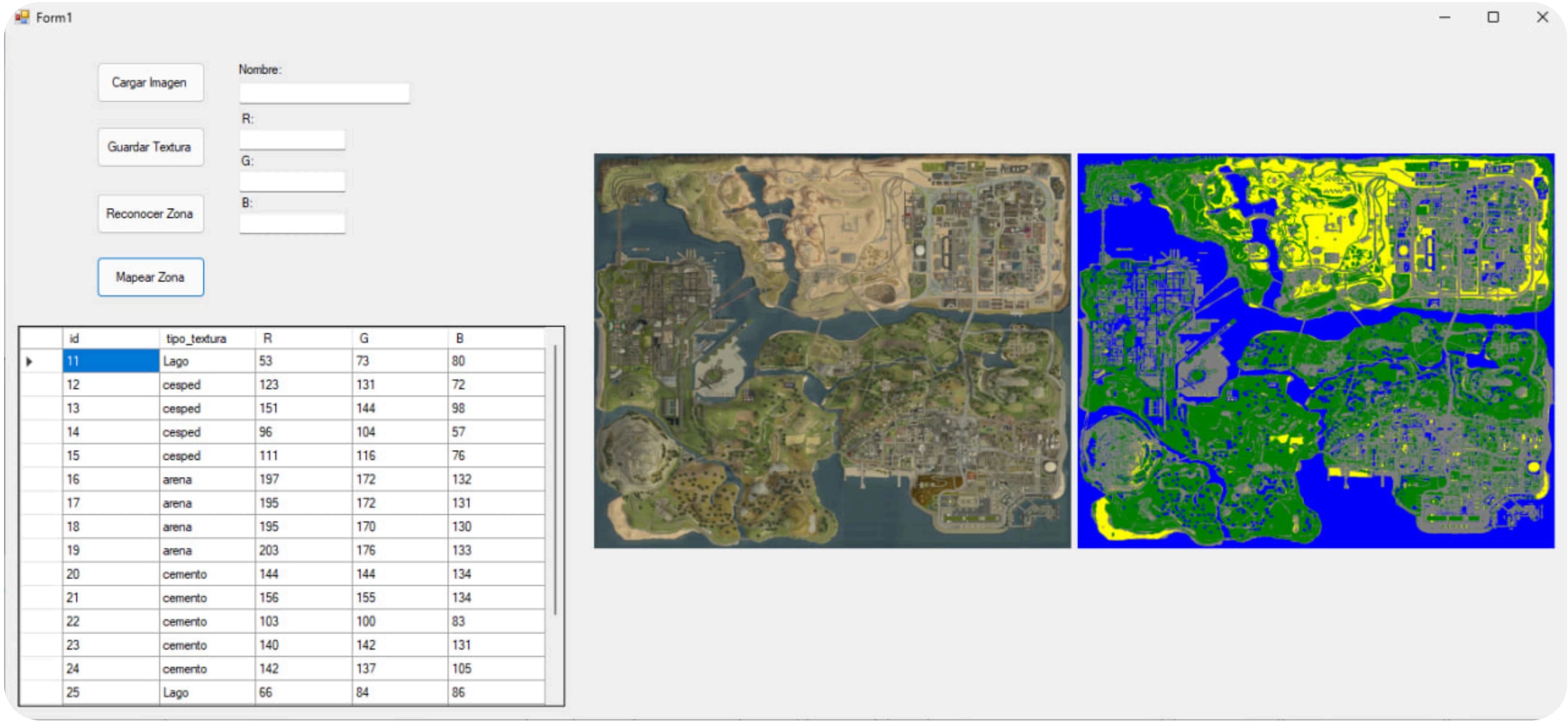
Finalmente, cada actividad fue sometida a pruebas funcionales para verificar el correcto funcionamiento de los algoritmos implementados y validar los resultados obtenidos respecto a los objetivos planteados.



4. ACTIVIDAD 1: CLASIFICACIÓN DE TEXTURAS

4.1 DESCRIPCIÓN GENERAL

La actividad consistió en desarrollar una aplicación capaz de identificar diferentes tipos de texturas presentes en una imagen digital mediante el análisis de los valores de color asociados a los píxeles seleccionados por el usuario. El sistema fue implementado utilizando Visual Basic y una base de datos para el almacenamiento de muestras de referencia.



4.2 METODOLOGÍA DE IMPLEMENTACIÓN

La aplicación dispone de funcionalidades para cargar imágenes, registrar nuevas muestras y reconocer texturas previamente almacenadas. Inicialmente, el usuario selecciona una imagen desde el sistema de archivos y la visualiza dentro de la interfaz.

Al hacer clic sobre una zona específica de la imagen, el sistema obtiene los valores correspondientes a los canales Rojo (R), Verde (G) y Azul (B) del píxel seleccionado. Estos valores son mostrados en la interfaz mediante cuadros de texto y pueden ser asociados a una descripción o nombre representativo de la textura observada.

Posteriormente, la información es almacenada en una base de datos, permitiendo construir un conjunto de muestras de referencia para diferentes tipos de superficies o regiones. Durante la fase de reconocimiento, el usuario selecciona nuevamente un píxel dentro de una imagen y el sistema compara sus componentes RGB con los registros almacenados.

A partir de dicha comparación, la aplicación determina la textura o zona que presenta mayor similitud respecto a las muestras previamente registradas y muestra el resultado correspondiente.

```
private void btnMapearZona_Click(object sender, EventArgs e)
{
    if (bmp == null)
        return;

    Bitmap mapa = new Bitmap(bmp.Width, bmp.Height);

    // Cargar texturas desde la BD
    List<Textura> texturas = new List<Textura>();

    using (SqlConnection conn = new SqlConnection(conexion))
    {
        conn.Open();

        string query = "SELECT tipo_textura, R, G, B FROM textura";

        using (SqlCommand cmd = new SqlCommand(query, conn))
        using (SqlDataReader dr = cmd.ExecuteReader())
        {
            while (dr.Read())
            {
                texturas.Add(new Textura()
                {
                    Tipo = dr.GetString(0),
                    R = dr.GetInt32(1),
                    G = dr.GetInt32(2),
                    B = dr.GetInt32(3)
                });
            }
        }

        // Analizar todos los pixeles
        for (int x = 0; x < bmp.Width; x++)
        {
            for (int y = 0; y < bmp.Height; y++)
            {
                Color pixel = bmp.GetPixel(x, y);

                string mejorTipo = "";
                double menorDistancia = double.MaxValue;

                foreach (var t in texturas)
                {
                    double distancia =
                        Math.Sqrt(
                            Math.Pow(pixel.R - t.R, 2) +
                            Math.Pow(pixel.G - t.G, 2) +
                            Math.Pow(pixel.B - t.B, 2));

                    if (distancia < menorDistancia)
                    {
                        menorDistancia = distancia;
                        mejorTipo = t.Tipo;
                    }
                }

                Color nuevoColor = Color.Black;
            }
        }
    }
}
```

4.3 TECNOLOGÍAS UTILIZADAS

- Visual Basic como lenguaje de programación.
- Windows Forms para la interfaz gráfica.
- Base de datos relacional para el almacenamiento de muestras.
- Manipulación de imágenes mediante objetos Bitmap.

4.4 RESULTADOS OBTENIDOS

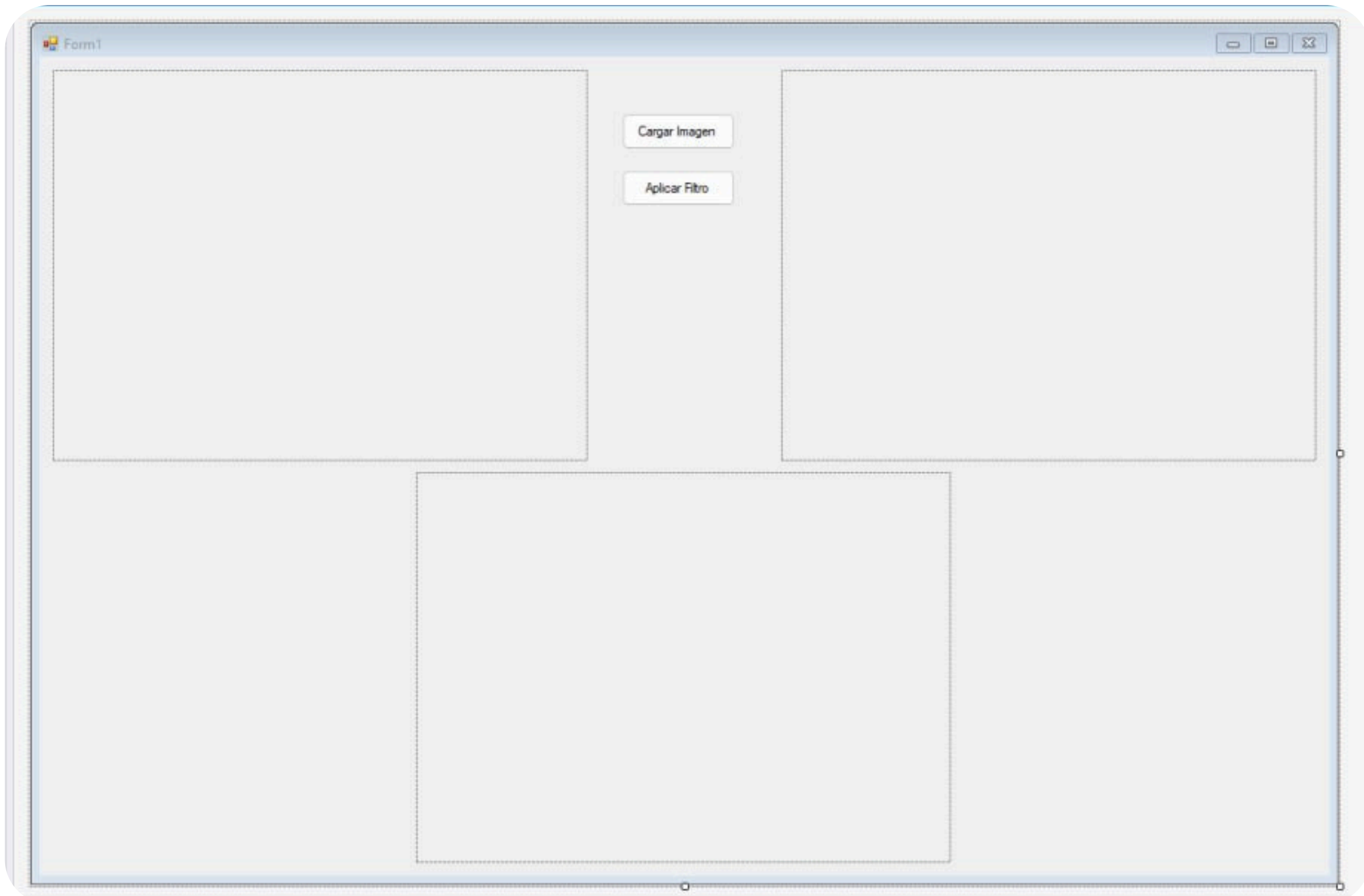
La aplicación permitió registrar y reconocer diferentes tipos de superficies utilizando información cromática obtenida directamente de los píxeles de una imagen. Los resultados obtenidos demostraron la viabilidad de utilizar características RGB como criterio básico de clasificación para sistemas de reconocimiento de texturas.

[illegible]

5. ACTIVIDAD 2: IMPLEMENTACIÓN DE UN FILTRO DE SUAVIZADO

5.1 DESCRIPCIÓN GENERAL

La actividad consistió en desarrollar una aplicación capaz de aplicar técnicas de suavizado sobre imágenes digitales mediante el análisis de bordes y procesamiento local de píxeles.



5.2 METODOLOGÍA DE IMPLEMENTACIÓN

La aplicación permite cargar una imagen digital y posteriormente ejecutar el proceso de suavizado mediante un botón específico de la interfaz.

Inicialmente se realiza una detección de bordes utilizando el operador de Laplace, el cual permite identificar cambios bruscos de intensidad entre píxeles vecinos. Para ello se emplean matrices de convolución que recorren la imagen evaluando la variación existente en cada región.

Una vez detectados los bordes, el sistema aplica un filtro de suavizado sobre las zonas identificadas. El procedimiento consiste en recorrer una ventana de 3×3 píxeles alrededor de cada borde encontrado y calcular nuevos valores representativos para la región analizada.

Este proceso permite reducir irregularidades visuales y generar transiciones más suaves entre píxeles, mejorando la apariencia general de la imagen sin eliminar completamente la información relevante.

5.3 TECNOLOGÍAS UTILIZADAS

- Visual Studio como entorno de desarrollo.
- Lenguaje de programación orientado a aplicaciones de escritorio.
- Bibliotecas de procesamiento de imágenes Bitmap.
- Operador de Laplace para detección de bordes.
- Filtros espaciales de suavizado mediante ventanas de 3×3 píxeles.

```
private void btnFiltro_Click(object sender, EventArgs e)
{
    Bitmap resultado = new Bitmap(
        imagenOriginal.Width,
        imagenOriginal.Height);

    Bitmap bordes = new Bitmap(
        imagenOriginal.Width,
        imagenOriginal.Height);

    // MATRIZ DE LAPLACE
    int[,] laplace =
    {
        { 0, 1, 0 },
        { 1, -4, 1 },
        { 0, 1, 0 }
    };

    int umbral = 10;

    for (int y = 3; y < imagenOriginal.Height - 3; y++)
    {
        for (int x = 3; x < imagenOriginal.Width - 3; x++)
        {
            int valorLaplace = 0;

            // DETECCIÓN DE BORDES
            for (int j = -1; j <= 1; j++)
            {
                for (int i = -1; i <= 1; i++)
                {
                    Color pixel =
                        imagenOriginal.GetPixel(x + i, y + j);

                    int gris =
                        (pixel.R + pixel.G + pixel.B) / 3;

                    valorLaplace +=
                        gris * laplace[j + 1, i + 1];
                }
            }

            valorLaplace = Math.Abs(valorLaplace);

            if (valorLaplace < 20)
            {
                valorLaplace = 0;
            }

            if (valorLaplace > 255)
            {
                valorLaplace = 255;
            }

            // INVERTIR
            valorLaplace = 255 - valorLaplace;

            Color colorBorde = Color.FromArgb(
                valorLaplace,
                valorLaplace,
                valorLaplace);

            bordes.SetPixel(x, y, colorBorde);

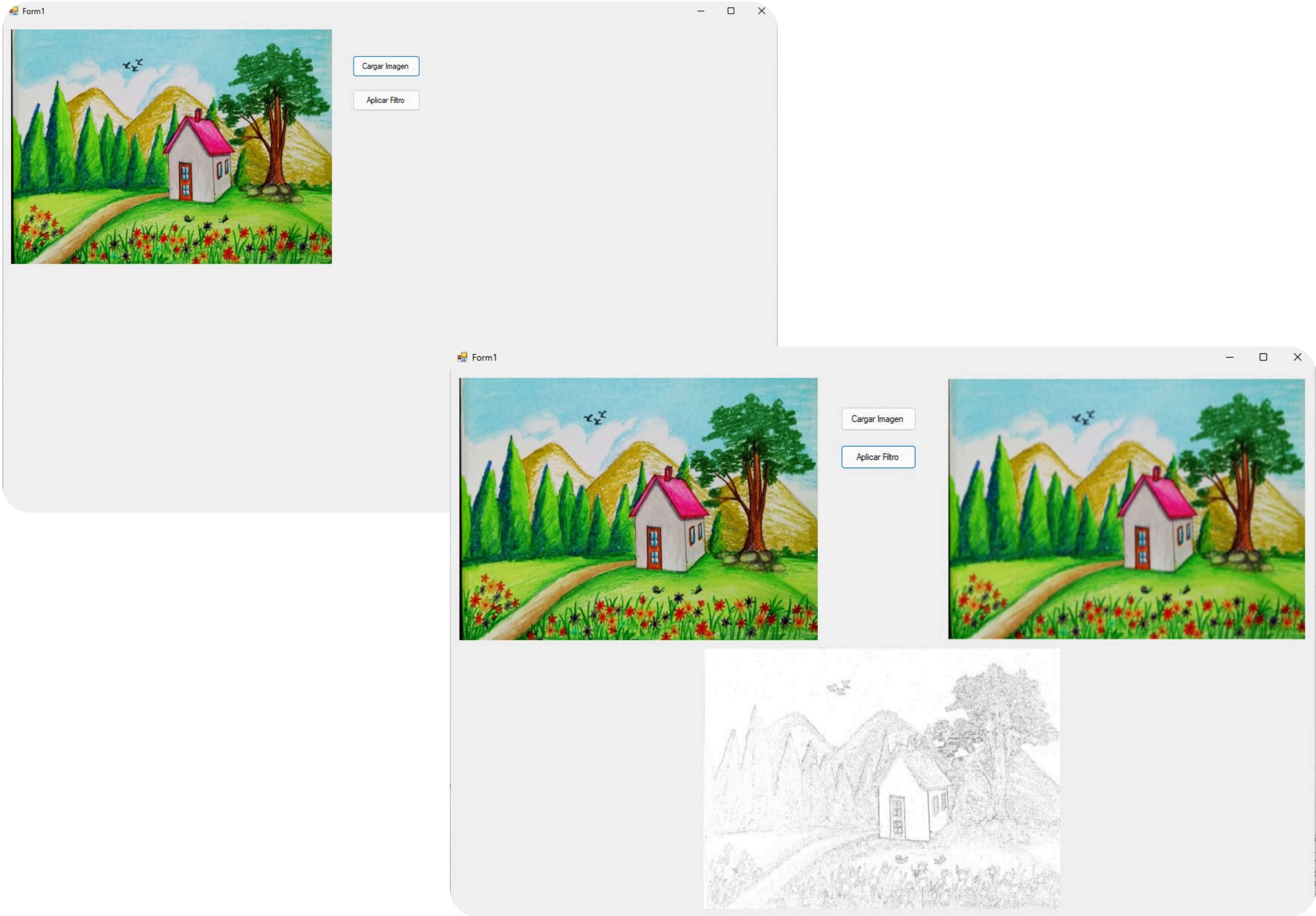
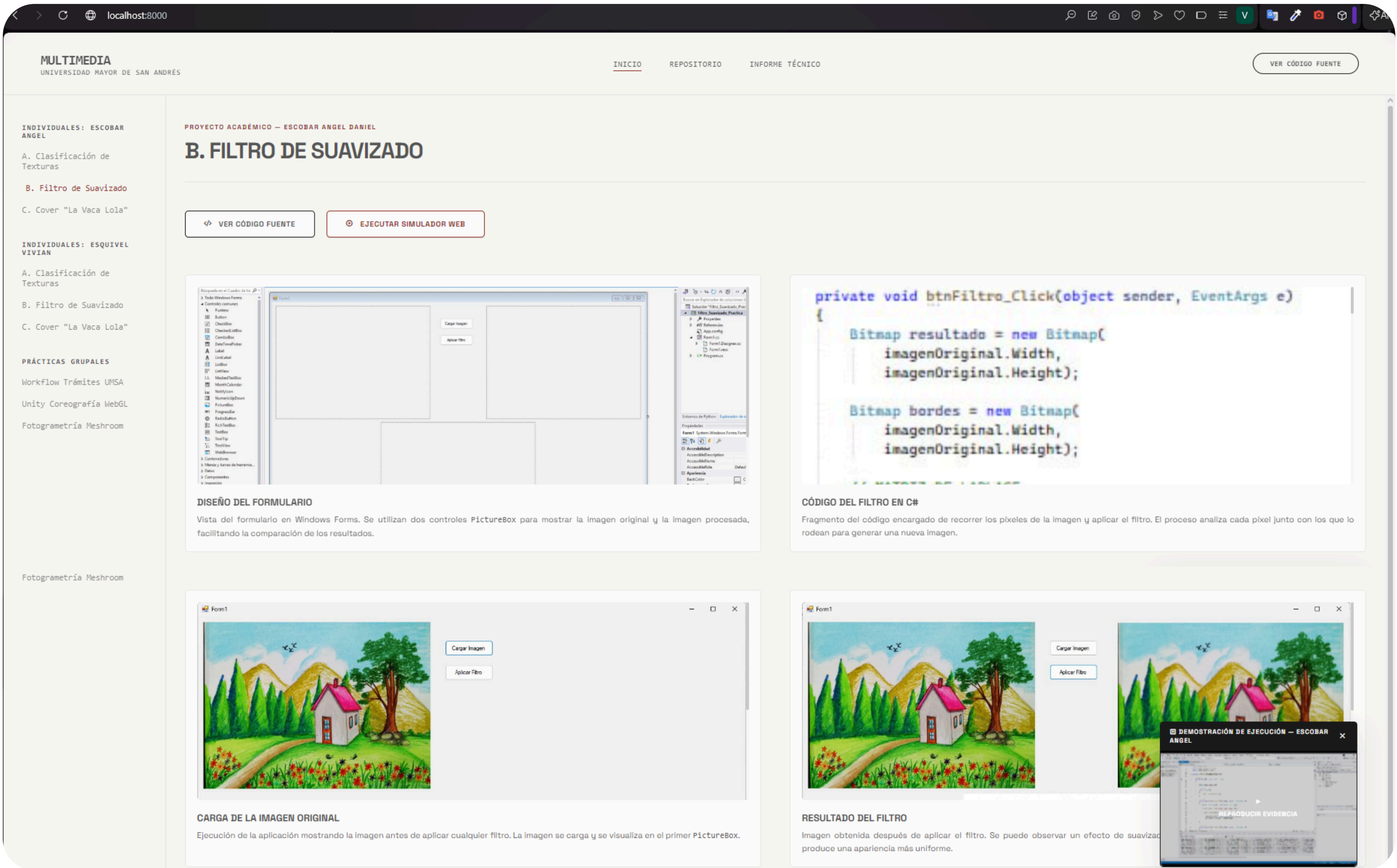
            bordes.SetPixel(x, y, colorBorde);

            // SI ES BORDE -> SUAVIZAR
            if (valorLaplace > umbral)
            {
                int sumaR = 0;
                int sumaG = 0;
                int sumaB = 0;
                int contador = 0;

                for (int j = -3; j <= 3; j++)
                {
```

5.4 RESULTADOS OBTENIDOS

La implementación permitió observar los efectos producidos por los filtros espaciales sobre imágenes digitales. La combinación entre detección de bordes y suavizado local contribuyó a reducir variaciones bruscas manteniendo una representación visual más uniforme de la imagen procesada.



6. ACTIVIDAD 3: PRODUCCIÓN MULTIMEDIA - COVER DE “LA VACA LOLA”

6.1 DESCRIPCIÓN GENERAL

La actividad consistió en desarrollar una producción multimedia basada en la canción infantil “La Vaca Lola”, integrando grabación de voz, procesamiento de audio mediante inteligencia artificial y animación tridimensional sincronizada en Unity.

6.2 METODOLOGÍA DE IMPLEMENTACIÓN

El proceso comenzó con la grabación de una interpretación propia de la canción utilizando un archivo de audio en formato WAV. Esta grabación sirvió como base para el posterior procesamiento de voz.

Posteriormente se empleó un modelo de conversión de voz basado en la tecnología Retrieval-Based Voice Conversion (RVC). Para ello se seleccionó un modelo correspondiente al personaje Homero Simpson, obteniendo los archivos necesarios para la inferencia del modelo, incluyendo los archivos de pesos y el índice de búsqueda utilizados por la red neuronal.

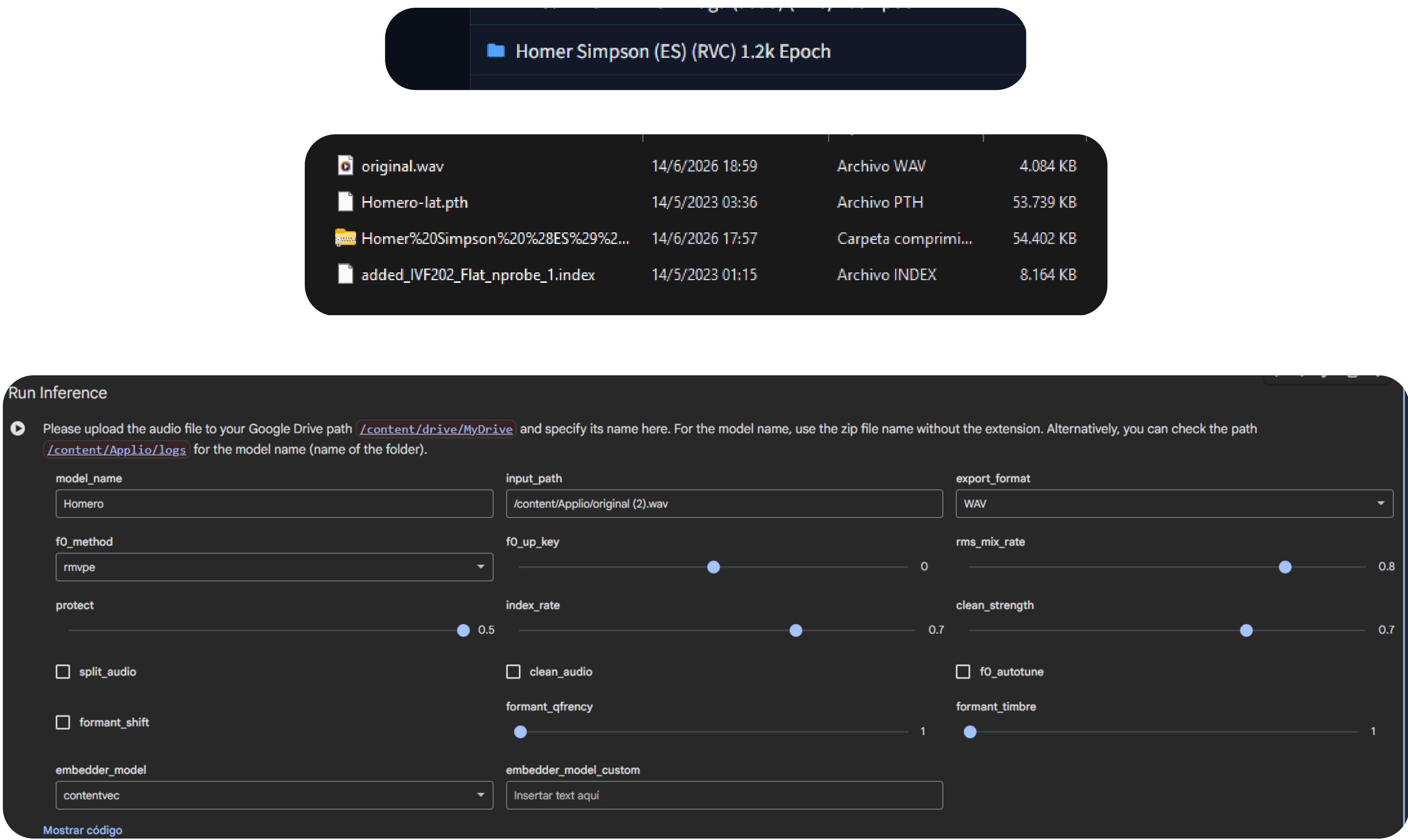
El procesamiento se realizó mediante la plataforma Applio ejecutada en Google Colab. Inicialmente se configuró el entorno de ejecución, instalando dependencias, bibliotecas y modelos requeridos por el sistema. Posteriormente se cargaron el archivo de audio original y los recursos asociados al modelo de voz seleccionado.

Una vez completada la configuración, se ejecutó el proceso de inferencia utilizando como entrada la grabación original realizada por el autor y como modelo de referencia la voz de Homero Simpson. Como resultado se generó una nueva versión de la canción con características vocales similares al personaje seleccionado.

Posteriormente, el audio generado fue importado dentro del motor Unity para la construcción de la escena multimedia. Se creó un componente de audio encargado de reproducir la canción generada y se configuró en modo de reproducción continua.

Para complementar la experiencia audiovisual se incorporaron personajes tridimensionales animados mediante secuencias obtenidas desde Mixamo. Las animaciones fueron configuradas para ejecutarse en bucle, sincronizándose con la reproducción de la canción procesada.

Finalmente, la escena fue ejecutada dentro de Unity obteniendo una representación multimedia donde los personajes realizan movimientos de baile mientras la canción es interpretada mediante la voz generada por inteligencia artificial.

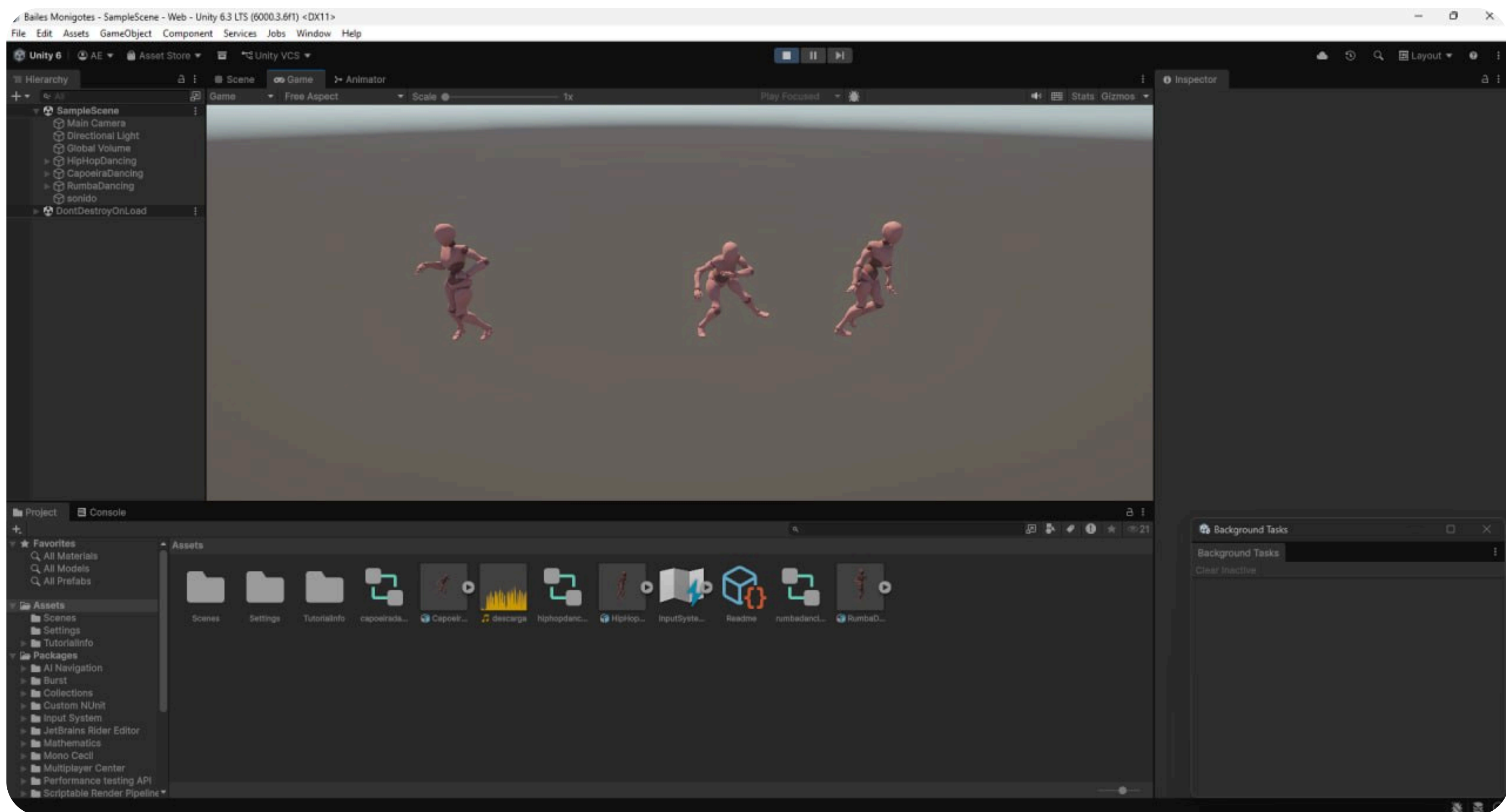


6.3 HERRAMIENTAS UTILIZADAS

- Google Colab para ejecución del entorno de inferencia.
- Applio para procesamiento de voz mediante inteligencia artificial.
- Hugging Face como repositorio de modelos de voz.
- Unity para el desarrollo de la escena tridimensional.
- Mixamo para la obtención de animaciones.

6.4 RESULTADOS OBTENIDOS

La actividad permitió integrar técnicas de grabación, procesamiento de audio, inteligencia artificial y animación tridimensional dentro de una misma producción multimedia. Como resultado se obtuvo una versión personalizada de la canción “La Vaca Lola”, interpretada mediante una voz generada digitalmente y acompañada por personajes animados sincronizados con la reproducción musical.



7. CONCLUSIONES

Las actividades desarrolladas permitieron aplicar conocimientos relacionados con procesamiento digital de imágenes, bases de datos, inteligencia artificial y producción multimedia. Mediante la implementación de algoritmos de análisis de imágenes fue posible comprender el funcionamiento de técnicas básicas de reconocimiento y filtrado utilizadas en aplicaciones de visión por computadora.

La actividad de clasificación de texturas permitió experimentar con el almacenamiento y comparación de características cromáticas, mientras que el filtro de suavizado evidenció la aplicación práctica de operadores espaciales para el mejoramiento de imágenes digitales.

Por otra parte, la producción multimedia permitió integrar herramientas modernas de conversión de voz mediante inteligencia artificial con motores gráficos tridimensionales, demostrando las posibilidades actuales de generación de contenido audiovisual asistido por software.

En conjunto, las actividades fortalecieron competencias relacionadas con programación, procesamiento digital de señales, desarrollo multimedia y utilización de tecnologías emergentes aplicadas a la creación de experiencias interactivas.